TradeMaster Cup 2025

4th Place Solution Report: Baseline ML → Leak Reconstruction → Tail Forecasting

Team: `zzz` (single-member team)
Author: `Baistan Zhyldyzbekov`

March 8, 2026

Abstract

This report documents my full competition path from an initial pure-ML approach to a structure-aware solution based on leakage reconstruction and tail-only forecasting. I submitted over 200 experiments and used that iteration budget to test hypotheses, not only hyperparameters. The final pipeline reconstructs targets deterministically where future `feature_16` values are observable, then applies fallback strategies only on the terminal unknown windows (last 10/60/240 rows for short/medium/long). I provide code-level proof for every core claim: dataset structure, leakage equations, test coverage, unknown-tail localization, boundary simulation, and submission behavior comparison.

Contents

1 Competition Context and Evaluation Rubric **3**
1.1 Evaluation Alignment (Compact) . . . 3
1.2 Rubric-to-Section Map . . . 3
1.3 Individual Contribution (100%) . . . 3

2 Creativity and Exploration: 200+ Submission Journey and Hypothesis Progression **3**
2.1 Phase 1: Pure ML Search . . . 4
2.2 Phase 2: Structural Suspicion . . . 4
2.3 Phase 3: Decomposition Into "Known" vs "Unknown" Regions . . . 4
2.4 Submission timeline (phase transitions) . . . 4
2.5 My Contribution in This Exploration . . . 4

3 Data Analysis Skills: Structural Audit **5**
3.1 Dataset shape and schema . . . 5
3.2 Time structure . . . 5
3.3 Cutoff phase consistency . . . 6
3.4 Missingness and numerical quality . . . 6
3.5 Structural findings summary . . . 6

4 Methods and Models: Initial Pure-ML Stage (XGBoost-Centric) **7**
4.1 Why this stage still mattered . . . 7
4.2 Proof artifact for baseline stage . . . 8

5 Data Analysis Skills: Leak Discovery and Mathematical Proof **8**
5.1 How the `-10` shift was identified . . . 8
5.2 Recovered target equations . . . 8
5.3 Numerical validation on train . . . 8
5.4 Leak feature scan evidence . . . 8

6 Methods and Models: Unknown Tail Isolation and Fallback Design **9**
6.1 End-to-end decomposition of the final solution . . . 9
6.2 Coverage on test (`n=34348`) . . . 9
6.3 Fallback candidates . . . 10
6.4 Naming used later in simulation tables . . . 10
6.5 Direct behavior comparison of two key models . . . 10

7 Presentation of Experiments: Unknown-Tail Day Simulation and Local Model Ranking **10**
7.1 Simulation idea . . . 11
7.2 Basic benchmark (ZERO, AR1, AR5) . . . 11
7.3 Advanced fallback comparison at cutoff 239 . . . 11
7.4 Statistical confidence (paired day-level comparisons) . . . 11
7.5 Interpretation . . . 12

8 Presentation of Experiments: Final Submissions and Private-LB Postmortem **12**
8.1 Known private-LB outcomes . . . 12
8.2 Consolidated Scoreboard . . . 12
8.3 Ablation view across stages (with score deltas) . . . 13
8.4 Final-Evaluation Slot Decision . . . 13
8.5 Final Selection Rationale . . . 13
8.6 Why local-CV ranking reversed on private LB . . . 13
8.7 Why `zero` beat selected tail models in private . . . 14
8.8 Model diversity check . . . 14
8.9 Main lesson from postmortem . . . 14
8.10 Threats to Validity . . . 14

9 Code Quality: Reproducibility and Engineering Practices **14**
9.1 Pipeline modularization . . . 14
9.2 Operational entry points and lightweight regression tests . . . 15
9.3 Environment and Dependencies . . . 15
9.4 Repository Snapshot . . . 15
9.5 Proof package (this report folder) . . . 15
9.6 Claim-to-Proof Traceability . . . 16
9.7 End-to-End Reproduction Commands . . . 16

10 Reflection and Future Work **17**
10.1 What worked . . . 17
10.2 What failed or underperformed . . . 17
10.3 Why the final slot selection failed . . . 17
10.4 What I would improve next time . . . 18
10.5 General lesson about evaluation design . . . 18

A Proof Outputs **18**

B Proof Code Listings **21**

C Core Pipeline Code Listings **41**

1 Competition Context and Evaluation Rubric

1.1 Evaluation Alignment (Compact)

The official final score has two parts: private leaderboard (50%) and report/code quality (50%). For the private-LB component, the host formula is:

$$\text{Score}_{\text{part1}} = 50 \cdot \left(1 - \frac{\text{rank} - 1}{\text{total teams} - 1}\right). \tag{1}$$

With `rank=4` and `total teams=57`:

$$\text{Score}_{\text{part1}} = 50 \cdot \left(1 - \frac{4 - 1}{57 - 1}\right) = 47.32. \tag{2}$$

The rest of this report focuses on the technical evidence supporting the report/code component.

1.2 Rubric-to-Section Map

Table 1: How this report maps to host rubric criteria

Rubric criterion	Main evidence sections
Data analysis skills (20%)	Sections 3, 5, 7 and Appendix A/B proof outputs and scripts
Code quality (20%)	Section 9, Appendix C core pipeline listings, sanity checks
Methods and models (20%)	Sections 4, 6, 7, 8 (baseline → leak → tail models)
Presentation of experiments & explanations (20%)	Sections 2, 7, 8 plus stage/ablation tables
Creativity (15%)	Sections 2, 5, 6 (problem reframing and exploitation strategy)
Reflection (5%)	Section 10 postmortem and future improvements

1.3 Individual Contribution (100%)

I competed as a single-member team (`zzz`). All work was completed by me:

- data analysis and diagnostics,
- model design and implementation,
- 200+ submission experimentation and tracking,
- leakage verification and tail-fallback design,
- final submission selection and report preparation.

Final outcome summary. I finished **4th place** as a single-member team and made **200+ submissions**. I used this budget for structured hypothesis testing under limited observability (public LB rounding to `0.0000` for many submissions).

2 Creativity and Exploration: 200+ Submission Journey and Hypothesis Progression

I started with standard supervised learning assumptions and treated this as a normal financial tabular forecasting competition. The development path had three phases.

2.1 Phase 1: Pure ML Search

I built an XGBoost-centered pipeline, tested feature engineering, and tried alternatives (CatBoost, LightGBM, neural-network attempts, and blends). This improved my score relative to basic baselines, but performance eventually plateaued.

2.2 Phase 2: Structural Suspicion

As public leaderboard scores became unusually low for multiple teams, I shifted from hyperparameter tuning to target-mechanism analysis. The key question became:

> Are targets generated from a deterministic transform of an observed test feature?

2.3 Phase 3: Decomposition Into "Known" vs "Unknown" Regions

After proving deterministic leakage, I reframed the task:

- Most rows can be reconstructed exactly (no ML needed there).
- Only the terminal tail requires true forecasting.

This decomposition explains why so many submissions were necessary: once scores are near zero and rounded on public LB, hidden decimals and tiny tail behavior differences dominate rank.

2.4 Submission timeline (phase transitions)

Early stage: baseline ML search (XGB, CatBoost, LightGBM, NN attempts).
$\Downarrow$
Middle stage: structural suspicion and leak equation testing (`feature_16`, lag scan, formula proof).
$\Downarrow$
Late stage: tail-only modeling and final-slot strategy under hidden-decimal uncertainty.

Figure 1: Milestone timeline across the 200+ submission process

The representative milestone log below summarizes the main hypothesis changes and decisions across the 200+ submission process.

2.5 My Contribution in This Exploration

The 200+ submissions were hypothesis-driven. For each stage, I documented what failed, updated assumptions, and narrowed the search space until the final decomposition (reconstruction + tail forecasting) was fully validated by both code and leaderboard behavior.

Table 2: Representative milestones from the 200+ submission path

Phase	Hypothesis	Action	Outcome / Decision
Early baseline	Strong tabular ML should beat naive fill	Trained pure XGBoost pipeline with feature engineering and chaining	Improved over host baseline; plateau near public-LB `0.773x-0.78x`
NN attempts	NN + tree blend may outperform trees alone	Tested MLP/ResNet-style variants and blends	Stayed around `0.803x`, near all-zero sample baseline; deprioritized NN
Suspicion stage	Scores near 0.0000 indicate structural weakness in assumptions	Switched from tuning to mechanism analysis	Identified deterministic leakage pattern candidates
Leak proof	Targets are generated from future `feature_16` transforms	Implemented shift/compound validation on train	Reconstruction error reduced to numerical precision
Tail isolation	Only a small terminal region is unknown	Quantified non-NaN coverage on test	Confirmed unknown windows are exactly last 10/60/240
Tail modeling	Tail priors can decide hidden decimals	Evaluated zero/AR/expanding/macro/robust via boundary simulation	Margins were small; required robust final selection strategy
Final evaluation	Best public 0.0000 may hide private decimal gaps	Chose final pair and analyzed post-release behavior	Hard-zero tail beat both selected non-zero tail priors on private (`0.0139` vs `0.0152/0.0153`)

3 Data Analysis Skills: Structural Audit

I ran a dedicated audit script:

- `report_overleaf/proofs/proof_01_dataset_structure.py`

I also include the original helper script used during the competition to verify cutoff-phase alignment:

- `notebooks/helper/verify_minute27_cutoff.py`

3.1 Dataset shape and schema

- Train: `(139392, 37)`
- Test: `(34348, 34)`
- Train has 3 target columns; test excludes targets.

3.2 Time structure

- Minute range is `0..239`.
- Test contains one partial opening day: `{582: 28}`.
- First test row: `(date_id=582, minute_id=212)`.
- Last test row: `(date_id=725, minute_id=239)`.

3.3 Cutoff phase consistency

The test size is 34348 rows, and:

- `34348 % 240 = 28`
- `(34348-1) % 240 = 27`

This means:

- the test set does not begin at minute 0 of a day;
- the last observed test row is raw `minute_id=239`;
- but relative to the hidden continuation, that row sits at cycle phase 27.

So there are two correct but different ways to describe the boundary:

- **raw-minute view:** the observed data ends at `minute_id=239`;
- **phase-aligned view:** the equivalent historical cutoff for backtesting is minute 27.

This is why both minute-27 and minute-239 boundaries were checked in later simulations.

3.4 Missingness and numerical quality

The dataset contains NaNs in several features (and one Inf in `feature_2`), but the key leakage variable and targets are numerically stable for formula validation windows:

- `train_feature_16`: 10 NaNs, 0 Infs
- `target_short/medium/long`: 0 NaNs, 0 Infs

3.5 Structural findings summary

Table 3: Summary of structural findings used by the final solution

Finding	How discovered	Proof script	Consequence for solution
Test starts mid-day	First test row audit	`P01`	Raw `minute_id` alone is not enough; phase alignment must be tracked.
Final observed row is raw `minute_id=239`	Last test row audit	`P01`	The hidden region is anchored at the end of the final day.
Aligned cutoff phase is 27	Test-length modulo-240 check and helper verification	`P01`, `P08`	Historical simulations should use minute-27 as the phase-matched boundary.
`target_short` is `feature_16.shift(-10)`	Lag sweep on train	`P02`	Short target can be reconstructed exactly whenever future `feature_16` is visible.
`target_medium/long` are compounded future `feature_16`	Formula validation on train	`P02`	Medium and long targets are also reconstructable except in terminal windows.
Only last `10/60/240` rows are unknown	Test coverage audit	`P03`	Only the final-day tail requires fallback forecasting.

Proof output is included in Appendix A.

4 Methods and Models: Initial Pure-ML Stage (XGBoost-Centric)

Before leak exploitation, I built a full machine-learning pipeline in:

- `notebooks/solution.py`

  Pipeline chronology (7 stages):

1. Environment setup and deterministic seeds.
2. Data loading and split initialization.
3. "Sniper" feature pipeline (scaling + engineered features).
4. "Refinery" stage (additional feature transformations).
5. Champion chained XGBoost training (target-by-target dependency).
6. Alternative models (Purist/Robust variants).
7. Ensemble blend (Refinery + Robust).

Here, "Sniper" denotes the main feature-engineering and scaling stage, "Refinery" adds cluster- and interaction-based features, "Champion" is the main chained XGBoost model, and "Purist" / "Robust" are alternative baseline variants used for comparison and ensemble diversity.

4.1 Why this stage still mattered

Even though the final solution depended more on leakage reconstruction, the pure-ML stage was important for:

- establishing a strong baseline,
- generating fallback candidates,
- understanding which model families were stable on this dataset.

Practical result: tree-based methods outperformed my NN attempts; CatBoost/LightGBM variants were not competitive enough versus XGBoost in this setting. In later tables, I denote the regenerated archival NN baseline as `nn_resnet_honest` and the best single pure-ML XGBoost baseline as `xgb_refinery`.

NN performance observation. I tested several NN variants (MLP/ResNet-style and NN-tree blends). After converting the archival notebook into `notebooks/archival_neural_baseline.py` and regenerating its submission, I observed that the output contained only four distinct 1e-6-rounded prediction triplets across all 34,348 test rows. Its leaderboard scores were also very close to the host `sample_submission.csv` baseline: `sample_submission.csv` scored `0.8031` public / `0.6121` private, while the archival NN baseline scored `0.8035` public / `0.6122` private. Based on these observations, my working hypothesis is that the tested NN variants converged to a near-constant, near-zero prediction pattern on this dataset, so I deprioritized them in later stages of the competition.

4.2 Proof artifact for baseline stage

To make this reproducible in the report package, I extracted the core baseline training blocks (champion, alternatives, ensemble) into:

- `report_overleaf/results/extracted_baseline_training_blocks.py`

using script:

- `report_overleaf/proofs/proof_07_extract_baseline_training_blocks.py`

5 Data Analysis Skills: Leak Discovery and Mathematical Proof

The central discovery was that all three targets are deterministic functions of future `feature_16`. I validated this directly on train using:

- `report_overleaf/proofs/proof_02_leak_formula_validation.py`

5.1 How the `-10` shift was identified

I did not assume `-10` first. I ran a lag sweep for `feature_16` against `target_short`, testing lags `1..60` (i.e., `shift(-lag)`):

- best lag: `10`, MAE $= 0$
- second best lag: `9`, MAE ≈ 0.00173234

This identifies an exact 10-minute displacement, not an approximate match.

5.2 Recovered target equations

For row index t:

$$\texttt{target_short}(t) = f_{16}(t + 10) \tag{3}$$

$$\texttt{target_medium}(t) = \prod_{k=1}^{6} (1 + f_{16}(t + 10k)) - 1 \tag{4}$$

$$\texttt{target_long}(t) = \prod_{k=1}^{24} (1 + f_{16}(t + 10k)) - 1 \tag{5}$$

5.3 Numerical validation on train

Within scored-valid ranges, the reconstruction error is effectively zero:

- short: MAE $= 0$, max abs error $= 0$
- medium: MAE $\approx 2.01 \times 10^{-16}$
- long: MAE $\approx 6.27 \times 10^{-16}$

  These are floating-point precision-level residuals, not modeling errors.

5.4 Leak feature scan evidence

After identifying lag `10`, I scanned `feature_1..feature_30` with `shift(-10)` against `target_short`. The best feature by a large margin was:

- `feature_16`: MAE $= 0$

All others had materially higher MAE. This supports `feature_16` as the direct deterministic leak source.

6 Methods and Models: Unknown Tail Isolation and Fallback Design

After leak reconstruction, only terminal rows remain unknown on test. I proved exact coverage using:

- `report_overleaf/proofs/proof_03_test_coverage_and_unknown_tail.py`

Combining the structural audit from Section 3 with the exact leak equations from Section 5 gives the final task decomposition used by the solution: most test targets can be reconstructed directly from future `feature_16`, and only the terminal `10/60/240`-row windows require fallback forecasting.

6.1 End-to-end decomposition of the final solution

The full logic of the final pipeline can be summarized in four linked steps:

1. **Locate the boundary correctly (Section 3).** The last visible test row has raw `minute_id=239`, but because the test set starts mid-day, the phase-aligned historical cutoff is minute 27.
2. **Reconstruct what is already determined (Section 5).** All three targets are deterministic functions of future `feature_16`, so wherever those future values are visible in test, the targets can be recovered exactly.
3. **Isolate the truly unknown region (this section).** After applying the leak equations, only the last `10/60/240` rows remain unknown for short/medium/long.
4. **Model only the remaining tail (Section 7).** Boundary simulation is then used to rank fallback priors for that final unknown region.

So the competition is not solved by forecasting all `34,348` test rows directly. It decomposes into exact reconstruction on the known region plus a much smaller tail-forecasting problem on the final day.

6.2 Coverage on test (`n=34348`)

- short: non-NaN 34338, NaN 10 (last 10 rows)
- medium: non-NaN 34288, NaN 60 (last 60 rows)
- long: non-NaN 34108, NaN 240 (last 240 rows)

Tail starts on the last date (`date_id=725`) as expected:

- long unknown region starts at minute 0 of final day
- medium unknown region starts at minute 180
- short unknown region starts at minute 230

6.3 Fallback candidates

I evaluated multiple ways to fill the unknown future `feature_16` horizon:

- hard zero,
- AR1 / AR5,
- expanding minute mean,
- macro specialist (ridge next-day curve),
- robust-anchor blend (macro + expanding).

6.4 Naming used later in simulation tables

To keep later tables short, I use the following model shorthand:

- `zero`: hard-zero future for the unknown 240-minute horizon.
- `expanding_all`: minute-of-day mean future built from historical `feature_16`.
- `macro_lam0p1`: macro-specialist ridge model with regularization $\lambda = 0.1$.
- `robust_anchor`: 75/25 blend of macro-specialist future and `expanding_all`.

Implementation scripts:

- `notebooks/generate_zero_submission.py`
- `notebooks/generate_tail_variants.py`
- `notebooks/generate_tailvar_macro_specialist.py`
- `notebooks/generate_tailvar_robust_anchor.py`

6.5 Direct behavior comparison of two key models

I compared:

- `zero`
- `expanding_all`

with script:

- `report_overleaf/proofs/proof_04_tail_submission_behavior.py`

Result: in unknown windows, `zero` is constant all-zero output; `expanding_all` is dynamic non-zero output.

7 Presentation of Experiments: Unknown-Tail Day Simulation and Local Model Ranking

To rank fallback models offline, I built a boundary simulator:

- `report_overleaf/proofs/proof_05_boundary_simulation.py`

For completeness and historical traceability, I also include my original helper script used during the competition:

- `notebooks/helper/compare_boundary_cv.py`

7.1 Simulation idea

For each eligible historical day, I emulate the test condition by cutting at a boundary minute and forecasting the next 240-minute `feature_16` horizon. Then I reconstruct short/medium/long targets from that horizon and compute weighted MAE:

$$\mathrm{MAE}_{\mathrm{weighted}} = 0.5 \cdot \mathrm{MAE}_{\mathrm{short}} + 0.3 \cdot \mathrm{MAE}_{\mathrm{medium}} + 0.2 \cdot \mathrm{MAE}_{\mathrm{long}}. \tag{6}$$

Here, cutoff 27 is the phase-aligned historical analogue of the observed test boundary, while cutoff 239 is the raw-minute endpoint of the final visible test day. The day-count column in the following tables refers to valid historical backtest days that survive the boundary-simulation filters, not to the total number of calendar or trading days in the competition.

7.2 Basic benchmark (ZERO, AR1, AR5)

Table 4: Boundary simulation results for basic methods

Cutoff	Method	Valid Backtest Days	Mean MAE	Median MAE
27	ZERO	579	0.00773736	0.00669703
27	AR1	579	0.00727845	0.00617553
27	AR5	579	0.00719046	0.00626656
239	ZERO	578	0.01441898	0.01135339
239	AR1	578	0.01455810	0.01142253
239	AR5	578	0.01456054	0.01154876

7.3 Advanced fallback comparison at cutoff 239

In the advanced table, `global` means the future prior is fitted once using all historical training days, while `causal` means it is refit using only information available up to the simulated boundary day. Thus:

- `expanding_all_global` uses the full-train minute mean curve,
- `expanding_all_causal` uses only the historical prefix up to the simulated cutoff,
- `macro_lam0p1_global` uses the macro ridge model with $\lambda = 0.1$ fitted on all historical day transitions,
- `macro_lam0p1_causal` uses the same macro model refit only on past day transitions available at that simulated time,
- `robust_anchor_global` and `robust_anchor_causal` are the corresponding 75/25 blends of macro specialist and `expanding_all`.

7.4 Statistical confidence (paired day-level comparisons)

To avoid over-reading tiny mean differences, I added paired day-level statistics (win rate and bootstrap CI for mean difference) from the same simulation days.

Interpretation:

- Among top non-zero methods, CIs include zero; their ordering is statistically weak in this local proxy.
- Against `zero`, all three top non-zero methods have positive CI-separated advantage in local simulation.

Table 5: Advanced model ranking (cutoff 239)

Method	Valid Backtest Days	Mean MAE	Median MAE
`macro_lam0p1_global`	578	0.01428740	0.01126519
`robust_anchor_global`	578	0.01429183	0.01125059
`expanding_all_global`	578	0.01432369	0.01117355
`robust_anchor_causal`	578	0.01433106	0.01113515
`macro_lam0p1_causal`	578	0.01433723	0.01108783
`expanding_all_causal`	578	0.01434218	0.01113030
`zero`	578	0.01441898	0.01135339

Table 6: Pairwise confidence summary at cutoff 239 (lower MAE is better)

Pair (A vs B)	Mean(A-B)	Win rate (A better)	95% CI for mean(A-B)
`macro_lam0p1_global` vs `robust_anchor_global`	-0.00000443	50.52%	[-0.00001863, 0.00001014]
`macro_lam0p1_global` vs `expanding_all_global`	-0.00003628	51.04%	[-0.00009324, 0.00002180]
`robust_anchor_global` vs `expanding_all_global`	-0.00003186	51.73%	[-0.00007478, 0.00001135]
`zero` vs `expanding_all_global`	0.00009529	44.29%	[0.00002681, 0.00016806]
`zero` vs `robust_anchor_global`	0.00012715	44.98%	[0.00004744, 0.00020784]
`zero` vs `macro_lam0p1_global`	0.00013157	45.67%	[0.00004533, 0.00022138]

7.5 Interpretation

Local simulation suggested macro/robust/expanding variants were slightly better than zero on average. However, the margin was very small, so private split regime differences could reverse ordering. This is exactly what happened on private leaderboard for one of the final comparisons.

8 Presentation of Experiments: Final Submissions and Private-LB Postmortem

8.1 Known private-LB outcomes

After private results release, the following private-LB outcomes were available and used for postmortem:

Table 7: Private-LB comparison

Submission (short name)	Public LB	Private LB
`zero`	0.0000	0.0139
`expanding_all`	0.0000	0.0152
`robust_anchor`	0.0000	0.0153

Although all were rounded to `0.0000` on public LB, private decimals separated them.

8.2 Consolidated Scoreboard

Note: Timestamps and `.csv` suffixes from original submission filenames were removed to keep model names short. Also, `sample_submission` is the host baseline file, while `nn_resnet_honest` refers to the regenerated archival NN baseline produced by `notebooks/archival_neural_baseline.py`. Here, `zero` denotes the leak-reconstruction submission whose unknown tail windows were filled with hard zeros.

Table 8: Scoreboard across key stages (representative models)

Submission (short name)	Public LB	Private LB	Notes / evidence source
`sample_submission`	0.8031	0.6121	Host-provided all-zero baseline
`nn_resnet_honest`	0.8035	0.6122	Archival NN baseline; regenerated output collapsed to four near-zero templates
`xgb_refinery`	0.7733	0.6288	Best single XGB in pure-ML stage; overfit to public split relative to private
`zero`	0.0000	0.0139	Leak reconstruction + hard-zero tail behavior; not selected for final evaluation
`expanding_all`	0.0000	0.0152	Tail fallback with expanding minute mean; selected for final evaluation
`robust_anchor`	0.0000	0.0153	Tail fallback blend (macro + expanding anchor); selected for final evaluation

Table 9: Stage-wise ablation with public/private deltas (lower is better)

Stage model (short name)	Public	Private	ΔPublic vs prev	ΔPrivate vs prev	Key decision
`sample_submission`	0.8031	0.6121	–	–	Baseline reference
`nn_resnet_honest`	0.8035	0.6122	+0.0004	+0.0001	NN deprioritized after near-zero collapse
`xgb_refinery`	0.7733	0.6288	-0.0302	+0.0166	Strong public, weak private transfer
`zero`	0.0000	0.0139	-0.7733	-0.6149	Structural leap via leakage reconstruction
`expanding_all`	0.0000	0.0152	+0.0000	+0.0013	Selected final-eval slot
`robust_anchor`	0.0000	0.0153	+0.0000	+0.0001	Selected final-eval slot

8.3 Ablation view across stages (with score deltas)

8.4 Final-Evaluation Slot Decision

At selection time (before private release), I used my two final-evaluation slots on `expanding_all` and `robust_anchor`. `zero` and the macro-specialist candidate with $\lambda = 0.1$ were **not** selected for final evaluation. After private release, the postmortem showed that hard-zero tail behavior (as in `zero`) generalized better than both selected tail priors (`expanding_all`, `robust_anchor`) on the checked hidden split.

8.5 Final Selection Rationale

Table 10: Why final models were selected (and what I learned afterward)

Candidate	Local tail proxy (cutoff-239 weighted boundary MAE)	Private LB (known)	Selection / interpretation
`expanding_all`	Better than `zero` in local proxy (∼0.01432)	0.0152	Selected for final evaluation due to strong local simulation performance
`robust_anchor`	Competitive local proxy (∼0.01429–0.01434 family)	0.0153	Selected for final evaluation as robust-diversity candidate
`zero`	`zero` baseline around 0.01442 at cutoff-239 proxy	0.0139	Not selected for final evaluation; post-release comparator that outperformed both selected models

8.6 Why local-CV ranking reversed on private LB

Local cutoff-239 simulation favored non-zero tail priors over `zero` by small but CI-supported margins (e.g., `zero` − `macro_lam0p1_global` mean diff $\approx +1.3157 \times 10^{-4}$, 95% CI $[4.533 \times 10^{-5}, 2.2138 \times 10^{-4}]$). However, private outcomes reversed this ordering:

- `zero` (hard-zero tail behavior): `0.0139`
- `expanding_all`: `0.0152`
- `robust_anchor`: `0.0153`

This indicates hidden-tail regime shift: the private tail day behaved closer to zero-centered continuation than to the historical average patterns that helped in local simulation.

8.7 Why `zero` beat selected tail models in private

Using `report_overleaf/proofs/proof_04_tail_submission_behavior.py`, I verified:

- `zero` uses hard-zero in all unknown windows.
- `expanding_all` injects dynamic non-zero minute-mean values.
- `robust_anchor` private score (`0.0153`) was also worse than `zero` (`0.0139`).

Therefore, on this private hidden split, the unknown tail behaved closer to zero-centered dynamics than the selected non-zero tail priors.

8.8 Model diversity check

I also computed pairwise correlation/MAE in unknown windows with:

- `report_overleaf/proofs/proof_06_submission_correlation.py`

This showed `robust_anchor` and macro-family variants were highly correlated in tail windows, while zero-style fallback was structurally different in short-tail behavior.

8.9 Main lesson from postmortem

Near-zero public scores can hide meaningful private differences. In this competition, tiny assumptions on the final unknown 240-minute horizon were enough to change rank.

8.10 Threats to Validity

- Public leaderboard rounding to `0.0000` hides meaningful decimal-level differences.
- Local boundary simulation is a proxy, not an exact private-LB emulator.
- Distribution shift in the hidden tail day can reverse local-CV ranking.
- Highly correlated submission pairs reduce effective diversification.

9 Code Quality: Reproducibility and Engineering Practices

9.1 Pipeline modularization

I separated the solution into independent scripts:

- Baseline pure-ML pipeline: `notebooks/solution.py`
- Archival neural baseline: `notebooks/archival_neural_baseline.py`
- Core zero-tail submission generator: `notebooks/generate_zero_submission.py`
- Tail variant generator: `notebooks/generate_tail_variants.py`

- Macro specialist: `notebooks/generate_tailvar_macro_specialist.py`
- Robust-anchor blend: `notebooks/generate_tailvar_robust_anchor.py`
- Sanity-check suite: `notebooks/helper/run_sanity_checks.py`
- Shared reusable logic: `trademaster_core/`

The `trademaster_core/` module stores shared utilities for deterministic target reconstruction, tail-future generation, submission IO, and path resolution, so the main scripts can remain thin command-line entrypoints.

9.2 Operational entry points and lightweight regression tests

To make the repository easier to inspect and rerun, I also added:

- a root `Makefile` with one-command targets for sanity checks, proofs, diagnostics, and submission generation;
- a `tests/` folder with regression tests for leak equations, cutoff alignment, unknown-tail sizes, submission schema, zero-submission generation, tail-model finiteness, and helper-script smoke runs;
- a report-code sync script (`scripts/sync_report_code.py`) that copies source files into `report_overleaf/code/`, so appendix mirrors are synchronized rather than maintained manually.

These additions do not change the competition logic, but they make the final codebase easier to inspect, test, and reproduce.

9.3 Environment and Dependencies

- Python: `3.13.0`
- Dependency file: `requirements.txt`
- Key packages: `numpy==2.4.1`, `pandas==2.3.3`, `scikit_learn==1.8.0`, `xgboost==3.1.3`, `catboost==1.2.8`, `lightgbm==4.6.0`, `torch==2.9.1`.

9.4 Repository Snapshot

- Repository: `https://github.com/baistangz/kaggle-trademaster-hkust.git`
- Branch: `main`
- Report/code snapshot commit: `a8653346158b57291516eabd0714c882161545d5`

9.5 Proof package (this report folder)

This report includes executable proof scripts under:

- `report_overleaf/proofs/`

  Run all proofs with:

```
bash report_overleaf/proofs/run_all_proofs.sh
```

  This generates:

- text outputs in `report_overleaf/results/`
- CSV tables in `report_overleaf/tables/`

9.6 Claim-to-Proof Traceability

Table 11: Mapping from core claims to executable proof artifacts

Claim	Script ID	Output ID
Dataset structure and cutoff phase facts	P01	O01
Deterministic leak formula and feature scan	P02	O02
Unknown-tail window coverage on test	P03	O03
Tail behavior comparison of submissions	P04	O04
Boundary simulation model ranking	P05	O05
Tail-window submission correlation	P06	O06
Legacy cutoff verifier output	P08	O08
Legacy boundary-CV helper output	P09	O09

Script ID map:

- P01 = report_overleaf/proofs/proof_01_dataset_structure.py
- P02 = report_overleaf/proofs/proof_02_leak_formula_validation.py
- P03 = report_overleaf/proofs/proof_03_test_coverage_and_unknown_tail.py
- P04 = report_overleaf/proofs/proof_04_tail_submission_behavior.py
- P05 = report_overleaf/proofs/proof_05_boundary_simulation.py
- P06 = report_overleaf/proofs/proof_06_submission_correlation.py
- P08 = report_overleaf/proofs/proof_08_legacy_verify_minute27_cutoff_output.py
- P09 = report_overleaf/proofs/proof_09_legacy_compare_boundary_cv_output.py

Output ID map:

- O01 = proof_01_dataset_structure.txt
- O02 = proof_02_leak_formula_validation.txt
- O03 = proof_03_test_coverage_and_unknown_tail.txt
- O04 = proof_04_tail_submission_behavior.txt
- O05 = proof_05_boundary_simulation.txt + boundary_*.csv
- O06 = proof_06_submission_correlation.txt
- O08 = proof_08_legacy_verify_minute27_cutoff_output.txt
- O09 = proof_09_legacy_compare_boundary_cv_output.txt

9.7 End-to-End Reproduction Commands

```
# Optional one-command entry points
make sanity
make tests
make proofs
make sync-report-code

# 0) Run sanity checks for formulas/coverage/schema
python3 notebooks/helper/run_sanity_checks.py

# 1) Regenerate all proof outputs for the report
bash report_overleaf/proofs/run_all_proofs.sh

# 2) Generate archival neural baseline
python3 notebooks/archival_neural_baseline.py
```

```
# 3) Generate zero submission (with configured fallback)
python3 notebooks/generate_zero_submission.py --skip-train-check

# 4) Generate tail variants
python3 notebooks/generate_tail_variants.py

# 5) Generate macro specialist candidate
python3 notebooks/generate_tailvar_macro_specialist.py \
  --ridge-lambda 0.1 \
  --clip-abs 0.1

# 6) Generate robust-anchor candidate
python3 notebooks/generate_tailvar_robust_anchor.py \
  --macro-weight 0.75 \
  --ridge-lambda 0.1 \
  --clip-abs 0.1
```

10 Reflection and Future Work

10.1 What worked

- Reframing the problem from full ML prediction to deterministic reconstruction + tail forecasting.
- Using large submission budget for explicit hypothesis testing, not random tuning.
- Keeping multiple tail priors active to hedge hidden-decimal and split-regime risk.

10.2 What failed or underperformed

- NN attempts were unstable on this dataset.
- Some local-CV winners did not transfer to private-LB winners.
- Public LB rounding at `0.0000` made model selection harder and increased decision uncertainty.

10.3 Why the final slot selection failed

The main selection mistake was not that local simulation was useless, but that I treated a tiny mean-MAE advantage as more trustworthy than it really was under hidden-decimal uncertainty. In my cutoff-239 proxy, the non-zero tail priors (`expanding_all`, `robust_anchor`, macro variants) looked slightly better than `zero`, but the margins were extremely small and the top non-zero methods were highly correlated. In hindsight, this meant I was choosing between models whose local ordering was fragile, while also leaving out the most structurally different candidate. A stronger final-slot policy would have explicitly balanced two objectives:

- best local proxy score,
- strongest structural diversification under public-LB rounding.

Under that decision rule, `zero` deserved more weight because it represented a qualitatively different tail assumption rather than only a slightly worse local score.

10.4 What I would improve next time

- Build an automated experiment registry from day 1 (config, seed, score, rationale).
- Add stronger stress tests for regime shifts in the hidden tail segment.
- Quantify submission diversity earlier using tail-window correlation diagnostics.
- Align local model-selection criteria more directly with decision risk under public-LB rounding, instead of optimizing only the smallest proxy MAE.

10.5 General lesson about evaluation design

The broader lesson is that evaluation design can change what "best model" means. When a leaderboard is rounded and the remaining uncertainty is concentrated in a tiny hidden region, average validation error is no longer the only relevant quantity. Robust model selection must also account for:

- whether candidate submissions are meaningfully different from each other,
- whether the hidden split could be governed by a different local regime,
- whether the public metric hides the true ranking behind rounding.

In other words, once scores are compressed near zero, selecting two final submissions becomes a risk-management problem as much as a pure optimization problem.

Personal takeaway. This competition was my first deep experience of how far careful data understanding can outperform brute-force model tuning. The 200+ submission path reflects a complete engineering cycle: baseline building, failure diagnosis, structural discovery, targeted experimentation, and postmortem verification.

A Proof Outputs

This appendix embeds raw outputs generated by executable proof scripts.

A.1 Dataset structure output

```
=== Dataset Structure Proof ===
train_shape=(139392, 37)
test_shape=(34348, 34)
train_columns=37
test_columns=34

=== Date/Minute Structure ===
train_unique_date_id=581
test_unique_date_id=144
train_minute_range=[0, 239]
test_minute_range=[0, 239]
test_partial_days={582: 28}
test_first_row=(date_id=582, minute_id=212)
test_last_row=(date_id=725, minute_id=239)

=== Cutoff Phase Alignment Check ===
len_test=34348
len_test_mod_240=28
(len_test_minus_1)_mod_240=27
minute_from_phase_last=239
minute_from_phase_next=0

=== NaN/Inf Audit ===
train: nan_count=1547, inf_count=1
```

```
test: nan_count=288, inf_count=0
train_feature_16: nan=10, inf=0, min=-0.0918118466899, max=0.0957703265396
train_target_short: nan=0, inf=0, min=-0.0918118466899, max=0.0957703265396
train_target_medium: nan=0, inf=0, min=-0.0868044515103, max=0.0956399437412
train_target_long: nan=0, inf=0, min=-0.131662269129, max=0.164725457571
```

A.2 Leak formula validation output

```
=== Leak Formula Validation on Train ===
short:  valid_rows=139382, mae=0, max_abs_err=0
medium: valid_rows=139332, mae=2.00659368962e-16, max_abs_err=9.86623976962e-16
long:   valid_rows=139152, mae=6.27007869896e-16, max_abs_err=2.51187959321e-15

=== Lag Sweep for feature_16 -> target_short ===
best_lag=10, mae=0, valid_rows=139382, gap_to_second_best=0.00173234233895 (second lag=9)
top_10_lags_by_mae:
  lag=10 mae=0 valid_rows=139382
  lag= 9 mae=0.00173234233895 valid_rows=139382
  lag=11 mae=0.00173235476202 valid_rows=139381
  lag= 8 mae=0.00245289240738 valid_rows=139382
  lag=12 mae=0.00245292496304 valid_rows=139380
  lag= 7 mae=0.00299827258633 valid_rows=139382
  lag=13 mae=0.00299830561785 valid_rows=139379
  lag= 6 mae=0.00346448411363 valid_rows=139382
  lag=14 mae=0.00346451255458 valid_rows=139378
  lag= 5 mae=0.00387723958816 valid_rows=139382

=== Candidate Single-Feature Leak Scan for target_short ===
top_10_by_mae_shift_minus_10:
  feature_16 mae=0 valid_rows=139382
  feature_28 mae=0.00357687424318 valid_rows=139382
  feature_22 mae=0.00361371302788 valid_rows=139382
  feature_20 mae=0.00375602574504 valid_rows=139382
  feature_29 mae=0.00396370135435 valid_rows=139362
  feature_14 mae=0.00407461696292 valid_rows=139362
  feature_23 mae=0.00408023628813 valid_rows=139382
  feature_13 mae=0.00473121165743 valid_rows=139363
  feature_2  mae=0.00497031089355 valid_rows=139382
  feature_30 mae=0.00545804980541 valid_rows=139362
```

A.3 Test coverage and unknown tail output

```
=== Leak Coverage on Test ===
test_rows=34348
target_short: non_na=34338, na=10, expected_tail=10
  na_index_range=[34338, 34347] first=(date_id=725, minute_id=230) last=(date_id=725, minute_id=239)
target_medium: non_na=34288, na=60, expected_tail=60
  na_index_range=[34288, 34347] first=(date_id=725, minute_id=180) last=(date_id=725, minute_id=239)
target_long: non_na=34108, na=240, expected_tail=240
  na_index_range=[34108, 34347] first=(date_id=725, minute_id=0) last=(date_id=725, minute_id=239)
```

A.4 Tail submission behavior output

```
=== Tail Behavior Comparison ===
A=submissions/submission_FINAL_LEAK_20260222_040009.csv
B=submissions/tailvar/submission_TAILVAR_EXPANDING_ALL_20260224_030331.csv

[target_short] last_10
A: all_zero=True constant=True zeros=10 std=0 mean=0
B: all_zero=False constant=False zeros=0 std=0.000299116533458 mean=-0.00114683991347
A_vs_B: mae=0.00114683991347 equal_count=0/10

[target_medium] last_60
A: all_zero=True constant=True zeros=60 std=0 mean=0
B: all_zero=False constant=False zeros=0 std=0.00132984508173 mean=-0.00256618771132
A_vs_B: mae=0.00256618771132 equal_count=0/60

[target_long] last_240
A: all_zero=True constant=True zeros=240 std=0 mean=0
B: all_zero=False constant=False zeros=0 std=0.00329393577414 mean=-0.00380510863096
A_vs_B: mae=0.00424744551055 equal_count=0/240
```

A.5 Boundary simulation output

```
=== Boundary Simulation: Basic (ZERO/AR1/AR5) ===
scenario  cutoff method  valid_backtest_days   mean_mae  median_mae    std_mae
   basic      27   zero                  579 0.00773736  0.00669703 0.00432937
   basic      27    ar1                  579 0.00727845  0.00617553 0.00421160
   basic      27    ar5                  579 0.00719046  0.00626656 0.00407919
   basic     239   zero                  578 0.01441898  0.01135339 0.01046828
   basic     239    ar1                  578 0.01455810  0.01142253 0.01057422
   basic     239    ar5                  578 0.01456054  0.01154876 0.01051690

=== Boundary Simulation: Advanced (Cutoff 239) ===
          scenario  cutoff               method  valid_backtest_days   mean_mae  median_mae    std_mae
advanced_cutoff239     239  macro_lam0p1_global                  578 0.01428740  0.01126519 0.01046549
advanced_cutoff239     239 robust_anchor_global                  578 0.01429183  0.01125059 0.01047960
advanced_cutoff239     239 expanding_all_global                  578 0.01432369  0.01117355 0.01052774
advanced_cutoff239     239 robust_anchor_causal                  578 0.01433106  0.01113515 0.01063695
advanced_cutoff239     239  macro_lam0p1_causal                  578 0.01433723  0.01108783 0.01063432
advanced_cutoff239     239 expanding_all_causal                  578 0.01434218  0.01113030 0.01064936
advanced_cutoff239     239                 zero                  578 0.01441898  0.01135339 0.01046828

=== Pairwise Confidence (Cutoff 239, day-level paired) ===
       a        b   n      mean_d       med_d      win_a       ci_lo      ci_hi
 macro_g robust_g 578 -0.00000443 -0.00000171 0.50519031 -0.00001863 0.00001014
 macro_g expand_g 578 -0.00003628 -0.00000932 0.51038062 -0.00009324 0.00002180
robust_g expand_g 578 -0.00003186 -0.00001009 0.51730104 -0.00007478 0.00001135
    zero expand_g 578  0.00009529  0.00026886 0.44290657  0.00002681 0.00016806
    zero robust_g 578  0.00012715  0.00015241 0.44982699  0.00004744 0.00020784
    zero  macro_g 578  0.00013157  0.00012694 0.45674740  0.00004533 0.00022138
```

A.6 Submission correlation output

```
=== Submission Correlation in Unknown Tail Windows ===

[target_short] last_10
final_leak   vs expanding_all | corr=nan | mae=0.00114684
final_leak   vs robust_anchor | corr=nan | mae=0.00149280
final_leak   vs macro_lam0p1 | corr=nan | mae=0.00160812
final_leak   vs zero_patched | corr=nan | mae=0.00000000
expanding_all vs robust_anchor | corr=0.978722 | mae=0.00034596
expanding_all vs macro_lam0p1 | corr=0.950161 | mae=0.00046128
expanding_all vs zero_patched | corr=nan | mae=0.00114684
robust_anchor vs macro_lam0p1 | corr=0.993914 | mae=0.00011532
robust_anchor vs zero_patched | corr=nan | mae=0.00149280
macro_lam0p1 vs zero_patched | corr=nan | mae=0.00160812

[target_medium] last_60
final_leak   vs expanding_all | corr=nan | mae=0.00256619
final_leak   vs robust_anchor | corr=nan | mae=0.00271784
final_leak   vs macro_lam0p1 | corr=nan | mae=0.00276842
final_leak   vs zero_patched | corr=nan | mae=0.00146701
expanding_all vs robust_anchor | corr=0.996629 | mae=0.00016083
expanding_all vs macro_lam0p1 | corr=0.994265 | mae=0.00021446
expanding_all vs zero_patched | corr=0.991210 | mae=0.00114939
robust_anchor vs macro_lam0p1 | corr=0.999687 | mae=0.00005363
robust_anchor vs zero_patched | corr=0.991198 | mae=0.00130104
macro_lam0p1 vs zero_patched | corr=0.989866 | mae=0.00135163

[target_long] last_240
final_leak   vs expanding_all | corr=nan | mae=0.00424745
final_leak   vs robust_anchor | corr=nan | mae=0.00447434
final_leak   vs macro_lam0p1 | corr=nan | mae=0.00455003
final_leak   vs zero_patched | corr=nan | mae=0.00355526
expanding_all vs robust_anchor | corr=0.998683 | mae=0.00026361
expanding_all vs macro_lam0p1 | corr=0.997660 | mae=0.00035152
expanding_all vs zero_patched | corr=0.996824 | mae=0.00086071
robust_anchor vs macro_lam0p1 | corr=0.999854 | mae=0.00008791
robust_anchor vs zero_patched | corr=0.996694 | mae=0.00112204
macro_lam0p1 vs zero_patched | corr=0.996069 | mae=0.00120919
```

A.7 Legacy helper output: verify_minute27_cutoff

```
=== Legacy Helper Output: verify_minute27_cutoff ===
=== Test Cutoff Verification ===
rows: 34348
first row: date=582, minute=212
last row : date=725, minute=239
rows % 240 = 28
(rows-1) % 240 = 27

Partial day structure:
{582: 28}

Phase alignment checks:
minute from phase_last = (first_minute + phase_last) % 240 = 239
minute from phase_next = (first_minute + phase_next) % 240 = 0

Interpretation:
- Raw test endpoint minute_id is 239.
- Equivalent cycle phase at endpoint is 27.
- Therefore minute-27 boundary simulation is the correct aligned cutoff for historical backtests.
```

A.8 Legacy helper output: compare_boundary_cv

```
=== Legacy Helper Output: compare_boundary_cv ===
Comparing cutoff minute 27 vs 239 on train boundary simulation...

cutoff=27 | valid_backtest_days=579
  ZERO mean=0.00773736 median=0.00669703 std=0.00432937
  AR1  mean=0.00727845 median=0.00617553 std=0.00421160
  AR5  mean=0.00719046 median=0.00626656 std=0.00407919

cutoff=239 | valid_backtest_days=578
  ZERO mean=0.01441898 median=0.01135339 std=0.01046828
  AR1  mean=0.01455810 median=0.01142253 std=0.01057422
  AR5  mean=0.01456054 median=0.01154876 std=0.01051690

Side-by-side mean MAE:
cutoff         27        239  delta_239_minus_27
method
ar5    0.00719046 0.01456054          0.00737008
ar1    0.00727845 0.01455810          0.00727965
zero   0.00773736 0.01441898          0.00668161
```

B Proof Code Listings

B.1 proof_01_dataset_structure.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd


def main() -> None:
    root = Path(__file__).resolve().parents[2]
    train = pd.read_csv(root / "data/raw/train_v2.csv")
    test = pd.read_csv(root / "data/raw/test_v2.csv")

    print("=== Dataset Structure Proof ===")
    print(f"train_shape={train.shape}")
    print(f"test_shape={test.shape}")
    print(f"train_columns={len(train.columns)}")
```

```python
    print(f"test_columns={len(test.columns)}")

    print("\n=== Date/Minute Structure ===")
    print(f"train_unique_date_id={train['date_id'].nunique()}")
    print(f"test_unique_date_id={test['date_id'].nunique()}")
    print(f"train_minute_range=[{int(train['minute_id'].min())}, {int(train['minute_id'].max())}]")
    print(f"test_minute_range=[{int(test['minute_id'].min())}, {int(test['minute_id'].max())}]")

    test_counts = test.groupby("date_id").size()
    partial_days = test_counts[test_counts != 240].to_dict()
    print(f"test_partial_days={partial_days}")

    first_date = int(test.iloc[0]["date_id"])
    first_min = int(test.iloc[0]["minute_id"])
    last_date = int(test.iloc[-1]["date_id"])
    last_min = int(test.iloc[-1]["minute_id"])
    print(f"test_first_row=(date_id={first_date}, minute_id={first_min})")
    print(f"test_last_row=(date_id={last_date}, minute_id={last_min})")

    n = len(test)
    phase_last = (n - 1) % 240
    phase_next = n % 240
    minute_from_phase_last = (first_min + phase_last) % 240
    minute_from_phase_next = (first_min + phase_next) % 240

    print("\n=== Cutoff Phase Alignment Check ===")
    print(f"len_test={n}")
    print(f"len_test_mod_240={n % 240}")
    print(f"(len_test_minus_1)_mod_240={phase_last}")
    print(f"minute_from_phase_last={minute_from_phase_last}")
    print(f"minute_from_phase_next={minute_from_phase_next}")

    print("\n=== NaN/Inf Audit ===")
    for name, df in [("train", train), ("test", test)]:
        numeric = df.select_dtypes(include=[np.number]).to_numpy(dtype="float64")
        print(
            f"{name}: nan_count={int(np.isnan(numeric).sum())}, "
            f"inf_count={int(np.isinf(numeric).sum())}"
        )

    for col in ["feature_16", "target_short", "target_medium", "target_long"]:
        if col not in train.columns:
            continue
        s = train[col].to_numpy(dtype="float64")
        print(
            f"train_{col}: nan={int(np.isnan(s).sum())}, inf={int(np.isinf(s).sum())}, "
            f"min={float(np.nanmin(s)):.12g}, max={float(np.nanmax(s)):.12g}"
        )


if __name__ == "__main__":
```

```
    main()
```

B.2 proof_02_leak_formula_validation.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path
import numpy as np
import pandas as pd


def compounded_target(feature_16: pd.Series, blocks: int) -> pd.Series:
    out = pd.Series(1.0, index=feature_16.index, dtype="float64")
    for k in range(1, blocks + 1):
        out *= 1.0 + feature_16.shift(-10 * k)
    return out - 1.0


def mae_with_mask(y_true: pd.Series, y_pred: pd.Series, mask: np.ndarray) -> tuple[float, float, int]:
    yt = y_true.to_numpy(dtype="float64")
    yp = y_pred.to_numpy(dtype="float64")
    valid = mask & np.isfinite(yt) & np.isfinite(yp)
    err = np.abs(yt[valid] - yp[valid])
    return float(err.mean()), float(err.max()), int(valid.sum())


def main() -> None:
    root = Path(__file__).resolve().parents[2]
    train = pd.read_csv(root / "data/raw/train_v2.csv")
    n = len(train)

    f16 = train["feature_16"]
    pred_short = f16.shift(-10)
    pred_medium = compounded_target(f16, 6)
    pred_long = compounded_target(f16, 24)

    mask_short = np.arange(n) < (n - 10)
    mask_medium = np.arange(n) < (n - 60)
    mask_long = np.arange(n) < (n - 240)

    mae_s, max_s, cnt_s = mae_with_mask(train["target_short"], pred_short, mask_short)
    mae_m, max_m, cnt_m = mae_with_mask(train["target_medium"], pred_medium, mask_medium)
    mae_l, max_l, cnt_l = mae_with_mask(train["target_long"], pred_long, mask_long)

    print("=== Leak Formula Validation on Train ===")
    print(f"short:  valid_rows={cnt_s}, mae={mae_s:.12g}, max_abs_err={max_s:.12g}")
    print(f"medium: valid_rows={cnt_m}, mae={mae_m:.12g}, max_abs_err={max_m:.12g}")
    print(f"long:   valid_rows={cnt_l}, mae={mae_l:.12g}, max_abs_err={max_l:.12g}")

```

```python
    print("\n=== Lag Sweep for feature_16 -> target_short ===")
    lag_rows: list[tuple[int, float, int]] = []
    for lag in range(1, 61):
        pred = f16.shift(-lag)
        mae, _, cnt = mae_with_mask(train["target_short"], pred, np.ones(n, dtype=bool))
        lag_rows.append((lag, mae, cnt))

    lag_rows.sort(key=lambda x: x[1])
    best_lag, best_mae, best_cnt = lag_rows[0]
    second_lag, second_mae, _ = lag_rows[1]
    print(
        f"best_lag={best_lag}, mae={best_mae:.12g}, valid_rows={best_cnt}, "
        f"gap_to_second_best={second_mae - best_mae:.12g} (second lag={second_lag})"
    )
    print("top_10_lags_by_mae:")
    for lag, mae, cnt in lag_rows[:10]:
        print(f"  lag={lag:>2d} mae={mae:.12g} valid_rows={cnt}")

    print("\n=== Candidate Single-Feature Leak Scan for target_short ===")
    maes = []
    for j in range(1, 31):
        col = f"feature_{j}"
        pred = train[col].shift(-best_lag)
        mae, _, cnt = mae_with_mask(train["target_short"], pred, np.ones(n, dtype=bool))
        maes.append((col, mae, cnt))

    maes.sort(key=lambda x: x[1])
    print(f"top_10_by_mae_shift_minus_{best_lag}:")
    for col, mae, cnt in maes[:10]:
        print(f"  {col:<10s} mae={mae:.12g} valid_rows={cnt}")


if __name__ == "__main__":
    main()
```

B.3 proof_03_test_coverage_and_unknown_tail.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path
import pandas as pd


def compounded_target(feature_16: pd.Series, blocks: int) -> pd.Series:
    out = pd.Series(1.0, index=feature_16.index, dtype="float64")
    for k in range(1, blocks + 1):
        out *= 1.0 + feature_16.shift(-10 * k)
    return out - 1.0


```

```python
def describe_tail(test: pd.DataFrame, pred: pd.Series, name: str, tail: int) -> None:
    na_idx = pred[pred.isna()].index
    print(f"{name}: non_na={pred.notna().sum()}, na={pred.isna().sum()}, expected_tail={tail}")
    if len(na_idx) > 0:
        first_i = int(na_idx.min())
        last_i = int(na_idx.max())
        first_row = test.iloc[first_i]
        last_row = test.iloc[last_i]
        print(
            f"  na_index_range=[{first_i}, {last_i}] "
            f"first=(date_id={int(first_row['date_id'])}, minute_id={int(first_row['minute_id'])}) "
            f"last=(date_id={int(last_row['date_id'])}, minute_id={int(last_row['minute_id'])})"
        )


def main() -> None:
    root = Path(__file__).resolve().parents[2]
    test = pd.read_csv(root / "data/raw/test_v2.csv", usecols=["date_id", "minute_id", "feature_16"])
    f16 = test["feature_16"]

    short = f16.shift(-10)
    medium = compounded_target(f16, 6)
    long_ = compounded_target(f16, 24)

    print("=== Leak Coverage on Test ===")
    print(f"test_rows={len(test)}")
    describe_tail(test, short, "target_short", 10)
    describe_tail(test, medium, "target_medium", 60)
    describe_tail(test, long_, "target_long", 240)


if __name__ == "__main__":
    main()
```

B.4 proof_04_tail_submission_behavior.py

```python
#!/usr/bin/env python3
from __future__ import annotations

import argparse
from pathlib import Path
import numpy as np
import pandas as pd

TARGET_COLS = ["target_short", "target_medium", "target_long"]
WINDOWS = {"target_short": 10, "target_medium": 60, "target_long": 240}


def load_sub(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)
    req = {"id", *TARGET_COLS}
    miss = req - set(df.columns)
```

```python
    if miss:
        raise ValueError(f"Missing columns in {path}: {sorted(miss)}")
    return df[["id", *TARGET_COLS]].copy()


def describe_window(x: np.ndarray) -> dict[str, float | int | bool]:
    same_as_prev = np.mean(np.isclose(x[1:], x[:-1])) if len(x) > 1 else 1.0
    return {
        "mean": float(np.mean(x)),
        "std": float(np.std(x)),
        "min": float(np.min(x)),
        "max": float(np.max(x)),
        "zeros": int(np.isclose(x, 0.0).sum()),
        "unique_rounded_12": int(len(np.unique(np.round(x, 12)))),
        "all_zero": bool(np.all(np.isclose(x, 0.0))),
        "constant": bool(np.all(np.isclose(x, x[0]))),
        "ffill_like_ratio": float(same_as_prev),
    }


def main() -> None:
    parser = argparse.ArgumentParser(description="Compare tail behavior of two submissions.")
    parser.add_argument(
        "--a",
        type=Path,
        default=Path("submissions/submission_FINAL_LEAK_20260222_040009.csv"),
    )
    parser.add_argument(
        "--b",
        type=Path,
        default=Path("submissions/tailvar/submission_TAILVAR_EXPANDING_ALL_20260224_030331.csv"),
    )
    args = parser.parse_args()

    a = load_sub(args.a)
    b = load_sub(args.b)
    if len(a) != len(b):
        raise ValueError("Submissions have different row counts.")

    print("=== Tail Behavior Comparison ===")
    print(f"A={args.a}")
    print(f"B={args.b}")

    n = len(a)
    for col in TARGET_COLS:
        w = WINDOWS[col]
        xa = a[col].iloc[n - w :].to_numpy(dtype="float64")
        xb = b[col].iloc[n - w :].to_numpy(dtype="float64")
        sa = describe_window(xa)
        sb = describe_window(xb)

        print(f"\n[{col}] last_{w}")
        print(
```

```
            f"A: all_zero={sa['all_zero']} constant={sa['constant']} zeros={sa['zeros']} "
            f"std={sa['std']:.12g} mean={sa['mean']:.12g}"
        )
        print(
            f"B: all_zero={sb['all_zero']} constant={sb['constant']} zeros={sb['zeros']} "
            f"std={sb['std']:.12g} mean={sb['mean']:.12g}"
        )
        print(
            f"A_vs_B: mae={float(np.mean(np.abs(xa-xb))):.12g} "
            f"equal_count={int(np.isclose(xa, xb).sum())}/{w}"
        )


if __name__ == "__main__":
    main()
```

B.5 proof_05_boundary_simulation.py

```
#!/usr/bin/env python3
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Callable

import numpy as np
import pandas as pd


@dataclass
class Env:
    tr: pd.DataFrame
    f16: np.ndarray
    minute: np.ndarray
    date: np.ndarray
    by_day: dict[int, np.ndarray]
    day_curves: dict[int, np.ndarray]


def load_env() -> Env:
    root = Path(__file__).resolve().parents[2]
    tr = pd.read_csv(root / "data/raw/train_v2.csv")
    f16 = tr["feature_16"].to_numpy(dtype="float64")
    minute = tr["minute_id"].to_numpy(dtype="int64")
    date = tr["date_id"].to_numpy(dtype="int64")
    by_day = {int(d): np.where(date == d)[0] for d in np.unique(date)}

    day_curves: dict[int, np.ndarray] = {}
    for d, idx in by_day.items():
        if len(idx) != 240:
            continue
        vals = f16[idx]
        if np.all(np.isfinite(vals)):
            day_curves[d] = vals.copy()

```

```python
    return Env(tr=tr, f16=f16, minute=minute, date=date, by_day=by_day, day_curves=day_curves)


def boundary_score_for_future(env: Env, e: int, fut: np.ndarray) -> float:
    tr, f16 = env.tr, env.f16
    err_short = []
    for i in range(e - 9, e + 1):
        err_short.append(abs(tr.at[i, "target_short"] - fut[i + 10 - e - 1]))

    err_medium = []
    for i in range(e - 59, e + 1):
        vals = []
        for k in range(1, 7):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_medium.append(abs(tr.at[i, "target_medium"] - pred))

    err_long = []
    for i in range(e - 239, e + 1):
        vals = []
        for k in range(1, 25):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_long.append(abs(tr.at[i, "target_long"] - pred))

    return 0.5 * np.mean(err_short) + 0.3 * np.mean(err_medium) + 0.2 * np.mean(err_long)


def fut_zero(_: int, env: Env) -> np.ndarray:
    return np.zeros(240, dtype="float64")


def fut_ar1(e: int, env: Env) -> np.ndarray:
    hist = env.f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-4000:] if len(hist) > 4000 else hist
    if len(hist) < 20:
        return np.zeros(240, dtype="float64")

    denom = float(np.dot(hist[:-1], hist[:-1]))
    a = float(np.dot(hist[1:], hist[:-1]) / denom) if denom > 1e-12 else 0.0
    a = float(np.clip(a, -0.995, 0.995))

    out = np.zeros(240, dtype="float64")
    cur = float(hist[-1])
    for i in range(240):
        cur = a * cur
        out[i] = cur
    return out


```

```python
def fut_ar5(e: int, env: Env) -> np.ndarray:
    hist = env.f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-6000:] if len(hist) > 6000 else hist
    p = 5
    if len(hist) < p + 20:
        return np.zeros(240, dtype="float64")

    y = hist[p:]
    x = np.column_stack([np.ones(len(y))] + [hist[p - i - 1 : len(hist) - i - 1] for i in range(p)])
    try:
        beta, *_ = np.linalg.lstsq(x, y, rcond=None)
    except np.linalg.LinAlgError:
        return np.zeros(240, dtype="float64")

    buf = list(hist[-p:])
    out = np.zeros(240, dtype="float64")
    for i in range(240):
        row = np.array([1.0] + buf[::-1], dtype="float64")
        nxt = float(row @ beta)
        nxt = float(np.clip(nxt, -0.1, 0.1))
        out[i] = nxt
        buf = buf[1:] + [nxt]
    return out


def minute_mean_from_prefix(env: Env, e: int) -> np.ndarray:
    hist = pd.DataFrame({"minute_id": env.minute[: e + 1], "feature_16": env.f16[: e + 1]})
    mm = hist.groupby("minute_id")["feature_16"].mean().reindex(range(240)).fillna(0.0).to_numpy(dtype="float64")
    return mm


def fut_expanding_all_causal(e: int, env: Env) -> np.ndarray:
    mm = minute_mean_from_prefix(env, e)
    last_minute = int(env.minute[e])
    future_minutes = (last_minute + np.arange(1, 241)) % 240
    return mm[future_minutes]


def fut_expanding_all_global(e: int, env: Env, mm_global: np.ndarray) -> np.ndarray:
    last_minute = int(env.minute[e])
    future_minutes = (last_minute + np.arange(1, 241)) % 240
    return mm_global[future_minutes]


def day_features(curve: np.ndarray) -> np.ndarray:
    o = float(curve[0])
    c = float(curve[-1])
    h = float(np.max(curve))
    l = float(np.min(curve))
    s = float(np.std(curve))
    m = float(np.mean(curve))
    am = float(np.mean(np.abs(curve)))
    r = h - l
```

```python
    return np.array([1.0, o, c, h, l, s, m, am, r], dtype="float64")


def fit_macro_beta(day_curves: dict[int, np.ndarray], day_limit: int | None, lam: float = 0.1) -> np.ndarray | None:
    days = sorted(day_curves.keys())
    if day_limit is not None:
        days = [d for d in days if d <= day_limit]

    day_set = set(days)
    x_rows, y_rows = [], []
    for d in days:
        nxt = d + 1
        if nxt in day_set:
            x_rows.append(day_features(day_curves[d]))
            y_rows.append(day_curves[nxt])

    if len(x_rows) < 5:
        return None

    x = np.stack(x_rows, axis=0)
    y = np.stack(y_rows, axis=0)
    xtx = x.T @ x
    beta = np.linalg.solve(xtx + lam * np.eye(xtx.shape[0], dtype="float64"), x.T @ y)
    return beta


def fut_macro_from_beta(curve_today: np.ndarray, beta: np.ndarray, clip_abs: float = 0.1) -> np.ndarray:
    pred = day_features(curve_today) @ beta
    return np.clip(pred, -clip_abs, clip_abs).astype("float64")


def build_eval_days(env: Env, cutoff_minute: int) -> list[int]:
    eval_days: list[int] = []
    for d, idx in env.by_day.items():
        if len(idx) <= cutoff_minute:
            continue
        e = int(idx[cutoff_minute])
        if e < 239 or e + 240 >= len(env.tr):
            continue

        pref = env.f16[idx[: cutoff_minute + 1]]
        if not np.all(np.isfinite(pref)):
            continue

        eval_days.append(int(d))
    return sorted(eval_days)


def score_methods_basic(env: Env) -> pd.DataFrame:
    rows = []
    for cutoff in [27, 239]:
        eval_days = build_eval_days(env, cutoff)
        for method_name, fn in [("zero", fut_zero), ("ar1", fut_ar1), ("ar5", fut_ar5)]:
            vals = []
```

```python
            for d in eval_days:
                e = int(env.by_day[d][cutoff])
                fut = fn(e, env)
                vals.append(boundary_score_for_future(env, e, fut))
            a = np.asarray(vals, dtype="float64")
            rows.append(
                {
                    "scenario": "basic",
                    "cutoff": cutoff,
                    "method": method_name,
                    "eval_days": len(eval_days),
                    "mean_mae": float(a.mean()),
                    "median_mae": float(np.median(a)),
                    "std_mae": float(a.std()),
                }
            )
    return pd.DataFrame(rows)


def score_methods_advanced_cutoff239(env: Env) -> pd.DataFrame:
    cutoff = 239
    eval_days = build_eval_days(env, cutoff)

    mm_global = (
        pd.DataFrame({"minute_id": env.minute, "feature_16": env.f16})
        .groupby("minute_id")["feature_16"]
        .mean()
        .reindex(range(240))
        .fillna(0.0)
        .to_numpy(dtype="float64")
    )
    beta_global = fit_macro_beta(env.day_curves, day_limit=None, lam
        =0.1)
    if beta_global is None:
        raise RuntimeError("Global macro beta fit failed.")

    beta_cache: dict[int, np.ndarray | None] = {}

    out = {
        "zero": [],
        "expanding_all_causal": [],
        "expanding_all_global": [],
        "macro_lam0p1_causal": [],
        "macro_lam0p1_global": [],
        "robust_anchor_causal": [],
        "robust_anchor_global": [],
    }

    for d in eval_days:
        idx = env.by_day[d]
        e = int(idx[cutoff])
        curve_today = env.day_curves.get(d)
        if curve_today is None:
            continue

        fut_zero_arr = fut_zero(e, env)
        fut_exp_c = fut_expanding_all_causal(e, env)
        fut_exp_g = fut_expanding_all_global(e, env, mm_global)
```

```python

        if d not in beta_cache:
            beta_cache[d] = fit_macro_beta(env.day_curves, day_limit=d, lam=0.1)
        beta_c = beta_cache[d]
        if beta_c is None:
            fut_mac_c = np.zeros(240, dtype="float64")
        else:
            fut_mac_c = fut_macro_from_beta(curve_today, beta_c, clip_abs=0.1)

        fut_mac_g = fut_macro_from_beta(curve_today, beta_global, clip_abs=0.1)

        fut_rob_c = 0.75 * fut_mac_c + 0.25 * fut_exp_c
        fut_rob_g = 0.75 * fut_mac_g + 0.25 * fut_exp_g

        out["zero"].append(boundary_score_for_future(env, e, fut_zero_arr))
        out["expanding_all_causal"].append(boundary_score_for_future(env, e, fut_exp_c))
        out["expanding_all_global"].append(boundary_score_for_future(env, e, fut_exp_g))
        out["macro_lam0p1_causal"].append(boundary_score_for_future(env, e, fut_mac_c))
        out["macro_lam0p1_global"].append(boundary_score_for_future(env, e, fut_mac_g))
        out["robust_anchor_causal"].append(boundary_score_for_future(env, e, fut_rob_c))
        out["robust_anchor_global"].append(boundary_score_for_future(env, e, fut_rob_g))

    rows = []
    for method, vals in out.items():
        a = np.asarray(vals, dtype="float64")
        rows.append(
            {
                "scenario": "advanced_cutoff239",
                "cutoff": 239,
                "method": method,
                "eval_days": len(vals),
                "mean_mae": float(a.mean()),
                "median_mae": float(np.median(a)),
                "std_mae": float(a.std()),
            }
        )

    df = pd.DataFrame(rows)
    score_map = {k: np.asarray(v, dtype="float64") for k, v in out.items()}
    return df, score_map


def bootstrap_mean_diff_ci(
    a: np.ndarray,
    b: np.ndarray,
    n_boot: int = 4000,
    seed: int = 42,
```

```python
) -> tuple[float, float]:
    """Return bootstrap 95% CI for mean(a-b) under paired resampling."""
    if len(a) != len(b):
        raise ValueError("Arrays must have the same length for paired CI.")
    n = len(a)
    if n == 0:
        return float("nan"), float("nan")

    rng = np.random.default_rng(seed)
    diffs = a - b
    boot = np.empty(n_boot, dtype="float64")
    for i in range(n_boot):
        idx = rng.integers(0, n, size=n)
        boot[i] = float(np.mean(diffs[idx]))
    lo, hi = np.quantile(boot, [0.025, 0.975])
    return float(lo), float(hi)


def pairwise_confidence_table(
    score_map: dict[str, np.ndarray],
    pairs: list[tuple[str, str]],
) -> pd.DataFrame:
    """Build pairwise day-level comparison table for selected methods."""
    rows = []
    for a_name, b_name in pairs:
        a = score_map[a_name]
        b = score_map[b_name]
        if len(a) != len(b):
            raise ValueError(f"Score length mismatch for pair {a_name} vs {b_name}")
        diff = a - b
        ci_lo, ci_hi = bootstrap_mean_diff_ci(a, b)
        rows.append(
            {
                "method_a": a_name,
                "method_b": b_name,
                "eval_days": len(diff),
                "mean_diff_a_minus_b": float(np.mean(diff)),
                "median_diff_a_minus_b": float(np.median(diff)),
                "win_rate_a_better": float(np.mean(a < b)),
                "tie_rate": float(np.mean(a == b)),
                "ci95_low": ci_lo,
                "ci95_high": ci_hi,
            }
        )
    return pd.DataFrame(rows)


def main() -> None:
    env = load_env()

    print("=== Boundary Simulation: Basic (ZERO/AR1/AR5) ===")
    basic = score_methods_basic(env)
    basic_print = basic.rename(columns={"eval_days": "valid_backtest_days"})
```

```python
    print(basic_print.to_string(index=False, float_format=lambda x: f"{x:.8f}"))

    print("\n=== Boundary Simulation: Advanced (Cutoff 239) ===")
    adv, score_map = score_methods_advanced_cutoff239(env)
    adv_print = adv.sort_values("mean_mae").rename(columns={"eval_days": "valid_backtest_days"})
    print(adv_print.to_string(index=False, float_format=lambda x: f"{x:.8f}"))

    pairs = [
        ("macro_lam0p1_global", "robust_anchor_global"),
        ("macro_lam0p1_global", "expanding_all_global"),
        ("robust_anchor_global", "expanding_all_global"),
        ("zero", "expanding_all_global"),
        ("zero", "robust_anchor_global"),
        ("zero", "macro_lam0p1_global"),
    ]
    pw = pairwise_confidence_table(score_map, pairs)
    print("\n=== Pairwise Confidence (Cutoff 239, day-level paired) ===")
    method_alias = {
        "macro_lam0p1_global": "macro_g",
        "robust_anchor_global": "robust_g",
        "expanding_all_global": "expand_g",
        "zero": "zero",
    }
    pw_print = pw.copy()
    pw_print["method_a"] = pw_print["method_a"].map(method_alias).fillna(pw_print["method_a"])
    pw_print["method_b"] = pw_print["method_b"].map(method_alias).fillna(pw_print["method_b"])
    pw_print = pw_print.rename(
        columns={
            "method_a": "a",
            "method_b": "b",
            "eval_days": "n",
            "mean_diff_a_minus_b": "mean_d",
            "median_diff_a_minus_b": "med_d",
            "win_rate_a_better": "win_a",
            "ci95_low": "ci_lo",
            "ci95_high": "ci_hi",
        }
    )
    pw_print = pw_print[["a", "b", "n", "mean_d", "med_d", "win_a", "ci_lo", "ci_hi"]]
    print(pw_print.to_string(index=False, float_format=lambda x: f"{x:.8f}"))

    out_dir = Path(__file__).resolve().parents[1] / "tables"
    out_dir.mkdir(parents=True, exist_ok=True)
    basic_print.to_csv(out_dir / "boundary_basic.csv", index=False)
    adv_print.to_csv(out_dir / "boundary_advanced_cutoff239.csv", index=False)
    pw.to_csv(out_dir / "boundary_pairwise_confidence_cutoff239.csv", index=False)


```

```python
if __name__ == "__main__":
    main()
```

B.6 proof_06_submission_correlation.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from itertools import combinations
from pathlib import Path
import numpy as np
import pandas as pd

SUBS = {
    "final_leak": "submissions/submission_FINAL_LEAK_20260222_040009.csv",
    "expanding_all": "submissions/tailvar/submission_TAILVAR_EXPANDING_ALL_20260224_030331.csv",
    "robust_anchor": "submissions/submission_TAILVAR_ROBUST_ANCHOR_W0p75_LAM0p1_20260224_214808.csv",
    "macro_lam0p1": "submissions/submission_TAILVAR_MACRO_SPECIALIST_LAM0p1_CHECK.csv",
    "zero_patched": "submissions/tailvar/submission_TAILVAR_ZERO_PATCHED_20260223_164342.csv",
}

WINDOWS = {
    "target_short": 10,
    "target_medium": 60,
    "target_long": 240,
}


def main() -> None:
    root = Path(__file__).resolve().parents[2]
    dfs = {k: pd.read_csv(root / v) for k, v in SUBS.items()}

    print("=== Submission Correlation in Unknown Tail Windows ===")
    for col, w in WINDOWS.items():
        print(f"\n[{col}] last_{w}")
        for a, b in combinations(dfs.keys(), 2):
            xa = dfs[a][col].iloc[-w:].to_numpy(dtype="float64")
            xb = dfs[b][col].iloc[-w:].to_numpy(dtype="float64")

            if np.std(xa) == 0 or np.std(xb) == 0:
                corr = float("nan")
            else:
                corr = float(np.corrcoef(xa, xb)[0, 1])

            mae = float(np.mean(np.abs(xa - xb)))
            print(f"{a:12s} vs {b:12s} | corr={corr:.6f} | mae={mae:.8f}")


if __name__ == "__main__":
    main()
```

B.7 proof_07_extract_baseline_training_blocks.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path


def extract_function_block(lines: list[str], func_name: str) -> list[str]:
    """Extract one top-level function block by name from source lines."""
    start = None
    marker = f"def {func_name}("
    for i, ln in enumerate(lines):
        if ln.startswith(marker):
            start = i
            break
    if start is None:
        raise ValueError(f"Function not found: {func_name}")

    end = len(lines)
    for j in range(start + 1, len(lines)):
        if lines[j].startswith("def "):
            end = j
            break
    return lines[start:end]


def main() -> None:
    """Build a baseline artifact from key training blocks in solution.py."""
    root = Path(__file__).resolve().parents[2]
    src_path = root / "notebooks/solution.py"
    lines = src_path.read_text().splitlines()
    keep_funcs = [
        "train_champion_refinery",
        "train_purist",
        "train_robust",
        "blend_refinery_and_robust",
    ]

    out_lines = []
    out_lines.append("# Extracted from notebooks/solution.py")
    out_lines.append("# Included as baseline ML proof artifact for report appendix")
    out_lines.append("")

    for func_name in keep_funcs:
        block = extract_function_block(lines, func_name)
        out_lines.append(f"# ===== BASELINE FUNCTION: {func_name} =====")
        out_lines.extend(block)
        out_lines.append("")

    out_dir = Path(__file__).resolve().parents[1] / "results"
    out_dir.mkdir(parents=True, exist_ok=True)
    out_path = out_dir / "extracted_baseline_training_blocks.py"
```

```
    out_path.write_text("\n".join(out_lines) + "\n")

    print(f"Wrote: {out_path}")


if __name__ == "__main__":
    main()
```

B.8 proof_08_legacy_verify_minute27_cutoff_output.py

```
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path

import pandas as pd


def main() -> None:
    # Legacy helper equivalent of notebooks/helper/verify_minute27_cutoff.py
    # with root path fixed for the report package.
    root = Path(__file__).resolve().parents[2]
    test = pd.read_csv(root / "data" / "raw" / "test_v2.csv", usecols=["date_id", "minute_id"])

    n = len(test)
    first_date = int(test.iloc[0]["date_id"])
    first_minute = int(test.iloc[0]["minute_id"])
    last_date = int(test.iloc[-1]["date_id"])
    last_minute = int(test.iloc[-1]["minute_id"])

    counts = test.groupby("date_id").size()
    partial_days = counts[counts != 240]

    phase_last = (n - 1) % 240
    phase_next = n % 240

    minute_from_phase_last = (first_minute + phase_last) % 240
    minute_from_phase_next = (first_minute + phase_next) % 240

    print("=== Legacy Helper Output: verify_minute27_cutoff ===")
    print("=== Test Cutoff Verification ===")
    print(f"rows: {n}")
    print(f"first row: date={first_date}, minute={first_minute}")
    print(f"last row : date={last_date}, minute={last_minute}")
    print(f"rows % 240 = {n % 240}")
    print(f"(rows-1) % 240 = {phase_last}")
    print()
    print("Partial day structure:")
    print(partial_days.to_dict())
    print()
    print("Phase alignment checks:")
    print(f"minute from phase_last = (first_minute + phase_last) % 240 = {minute_from_phase_last}")
    print(f"minute from phase_next = (first_minute + phase_next) % 240 = {minute_from_phase_next}")
```

```python
    print()
    print("Interpretation:")
    print("- Raw test endpoint minute_id is 239.")
    print("- Equivalent cycle phase at endpoint is 27.")
    print("- Therefore minute-27 boundary simulation is the correct aligned cutoff for historical backtests.")


if __name__ == "__main__":
    main()
```

B.9 proof_09_legacy_compare_boundary_cv_output.py

```python
#!/usr/bin/env python3
from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd


# Legacy helper equivalent of notebooks/helper/compare_boundary_cv.py
# with root path fixed for the report package.
def fut_zero(_: int, f16: np.ndarray) -> np.ndarray:
    return np.zeros(240, dtype="float64")


def fut_ar1(e: int, f16: np.ndarray) -> np.ndarray:
    hist = f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-4000:] if len(hist) > 4000 else hist
    if len(hist) < 20:
        return np.zeros(240, dtype="float64")

    denom = float(np.dot(hist[:-1], hist[:-1]))
    a = float(np.dot(hist[1:], hist[:-1]) / denom) if denom > 1e-12 else 0.0
    a = float(np.clip(a, -0.995, 0.995))

    out = np.zeros(240, dtype="float64")
    cur = float(hist[-1])
    for i in range(240):
        cur = a * cur
        out[i] = cur
    return out


def fut_ar5(e: int, f16: np.ndarray) -> np.ndarray:
    hist = f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-6000:] if len(hist) > 6000 else hist
    p = 5
    if len(hist) < p + 20:
        return np.zeros(240, dtype="float64")

    y = hist[p:]
```

```python
    x = np.column_stack([np.ones(len(y))] + [hist[p - i - 1 : len(hist) - i - 1] for i in range(p)])
    try:
        beta, *_ = np.linalg.lstsq(x, y, rcond=None)
    except np.linalg.LinAlgError:
        return np.zeros(240, dtype="float64")

    buf = list(hist[-p:])
    out = np.zeros(240, dtype="float64")
    for i in range(240):
        row = np.array([1.0] + buf[::-1], dtype="float64")
        nxt = float(row @ beta)
        nxt = float(np.clip(nxt, -0.1, 0.1))
        out[i] = nxt
        buf = buf[1:] + [nxt]
    return out


def boundary_score_for_future(tr: pd.DataFrame, f16: np.ndarray, e: int, fut: np.ndarray) -> float:
    err_short = []
    for i in range(e - 9, e + 1):
        err_short.append(abs(tr.at[i, "target_short"] - fut[i + 10 - e - 1]))

    err_medium = []
    for i in range(e - 59, e + 1):
        vals = []
        for k in range(1, 7):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_medium.append(abs(tr.at[i, "target_medium"] - pred))

    err_long = []
    for i in range(e - 239, e + 1):
        vals = []
        for k in range(1, 25):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_long.append(abs(tr.at[i, "target_long"] - pred))

    return 0.5 * np.mean(err_short) + 0.3 * np.mean(err_medium) + 0.2 * np.mean(err_long)


def build_eval_indices(
    tr: pd.DataFrame, f16: np.ndarray, by_day: dict[int, np.ndarray], cutoff_minute: int
) -> list[int]:
    eval_idx: list[int] = []
    for d, idx in by_day.items():
        if len(idx) <= cutoff_minute:
            continue

        e = int(idx[cutoff_minute])
        if e < 239:
```

```python
            continue
        if e + 240 >= len(tr):
            continue

        pref = f16[idx[: cutoff_minute + 1]]
        if not np.all(np.isfinite(pref)):
            continue

        eval_idx.append(e)
    return eval_idx


def run_cutoff(
    tr: pd.DataFrame, f16: np.ndarray, by_day: dict[int, np.ndarray], cutoff_minute: int
) -> tuple[int, dict[str, dict[str, float]]]:
    methods = {
        "zero": fut_zero,
        "ar1": fut_ar1,
        "ar5": fut_ar5,
    }
    eval_idx = build_eval_indices(tr, f16, by_day, cutoff_minute)

    out: dict[str, dict[str, float]] = {}
    for name, fn in methods.items():
        scores = []
        for e in eval_idx:
            fut = fn(e, f16)
            s = boundary_score_for_future(tr, f16, e, fut)
            scores.append(s)
        arr = np.asarray(scores, dtype="float64")
        out[name] = {
            "mean_mae": float(arr.mean()),
            "median_mae": float(np.median(arr)),
            "std_mae": float(arr.std()),
        }
    return len(eval_idx), out


def main() -> None:
    root = Path(__file__).resolve().parents[2]
    tr = pd.read_csv(root / "data" / "raw" / "train_v2.csv")
    f16 = tr["feature_16"].to_numpy(dtype="float64")
    date = tr["date_id"].to_numpy()
    by_day = {int(d): np.where(date == d)[0] for d in np.unique(date)}

    rows = []
    print("=== Legacy Helper Output: compare_boundary_cv ===")
    print("Comparing cutoff minute 27 vs 239 on train boundary simulation...")
    for cutoff in [27, 239]:
        n_days, result = run_cutoff(tr, f16, by_day, cutoff)
        print(f"\ncutoff={cutoff} | valid_backtest_days={n_days}")
        for method, stats in result.items():
            print(
                f"  {method.upper():<4} mean={stats['mean_mae']:.8f} "
                f"median={stats['median_mae']:.8f} std={stats['std_mae']:.8f}"
```

```
            )
            rows.append(
                {
                    "cutoff": cutoff,
                    "method": method,
                    "valid_backtest_days": n_days,
                    "mean_mae": stats["mean_mae"],
                    "median_mae": stats["median_mae"],
                    "std_mae": stats["std_mae"],
                }
            )

    df = pd.DataFrame(rows)
    pivot = df.pivot(index="method", columns="cutoff", values="mean_mae")
    if 27 in pivot.columns and 239 in pivot.columns:
        pivot["delta_239_minus_27"] = pivot[239] - pivot[27]
    print("\nSide-by-side mean MAE:")
    print(pivot.sort_values(by=27).to_string(float_format=lambda x: f"{x:.8f}"))


if __name__ == "__main__":
    main()
```

C Core Pipeline Code Listings

This appendix includes the exact scripts used to generate final submissions and baseline artifacts. Shared imported utilities are mirrored under `code/trademaster_core/` via `scripts/sync_report_code.py`.

C.1 generate_zero_submission.py

```
#!/usr/bin/env python3
from __future__ import annotations

"""Generate the zero-model submission.

This script reconstructs targets directly from `feature_16` whenever future
values are observable in test. Remaining NaNs at the tail are filled from a
configured fallback submission (or with zeros if fallback is disabled).

Submission naming convention for manual output names in docs/comments:
`submission_<PIPELINE>_<VARIANT>_CV<LOCAL_CV>.csv`
"""

import argparse
import sys
from datetime import datetime
from pathlib import Path

import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[1]
if str(REPO_ROOT) not in sys.path:
```

```python
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.constants import SUBMISSION_NAMING_CONVENTION, TARGET_COLS
from trademaster_core.leak_math import (
    build_known_region_predictions,
    compounded_target,
    known_region_counts,
)
from trademaster_core.submission_io import apply_fallback, load_submission, schema_aligned_submission


def validate_on_train(train_df: pd.DataFrame) -> None:
    """Print sanity metrics that verify the leak equations on train."""
    f16 = train_df["feature_16"]
    n = len(train_df)

    pred_short = f16.shift(-10)
    pred_medium = compounded_target(f16, 6)
    pred_long = compounded_target(f16, 24)

    mask_short = pd.Series(range(n)) < (n - 10)
    mask_medium = pd.Series(range(n)) < (n - 60)
    mask_long = pd.Series(range(n)) < (n - 240)

    mae_short = (train_df.loc[mask_short, "target_short"] - pred_short[mask_short]).abs().mean()
    mae_medium = (train_df.loc[mask_medium, "target_medium"] - pred_medium[mask_medium]).abs().mean()
    mae_long = (train_df.loc[mask_long, "target_long"] - pred_long[mask_long]).abs().mean()

    print("Train sanity check (scored ranges):")
    print(f"  short MAE  : {mae_short:.12g}")
    print(f"  medium MAE : {mae_medium:.12g}")
    print(f"  long MAE   : {mae_long:.12g}")

    byday_short = train_df.groupby("date_id")["feature_16"].shift(-10)
    missing_in_scored = int(byday_short[mask_short].isna().sum())
    byday_zero_mae = (train_df.loc[mask_short, "target_short"] - byday_short[mask_short].fillna(0.0)).abs().mean()
    print(f"  short rows lost by by-day shift(-10) in scored range: {missing_in_scored}")
    print(f"  short MAE if those rows are zero-filled: {byday_zero_mae:.12g}")

def parse_args(root: Path) -> argparse.Namespace:
    """Parse command-line arguments for submission generation."""
    parser = argparse.ArgumentParser(description="Generate zero submission for TradeMaster Cup 2025.")
    parser.add_argument("--test", type=Path, default=root / "data/raw/test_v2.csv")
    parser.add_argument("--train", type=Path, default=root / "data/raw/train_v2.csv")
    parser.add_argument("--sample", type=Path, default=root / "data/raw/sample_submission.csv")
    parser.add_argument(
```

```python
        "--fallback",
        type=Path,
        default=root / "data/raw/sample_submission.csv",
        help="Optional fallback submission for tail NaNs. Defaults to the host all-zero sample submission.",
    )
    parser.add_argument("--output-dir", type=Path, default=root / "submissions")
    parser.add_argument(
        "--output-name",
        type=str,
        default=None,
        help=(
            "Optional output filename. Recommended format: "
            "submission_ZERO_<VARIANT>_CV<LOCAL_CV>.csv"
        ),
    )
    parser.add_argument("--skip-train-check", action="store_true")
    return parser.parse_args()


def main() -> None:
    """Entry point."""
    root = REPO_ROOT
    args = parse_args(root)

    test_df = pd.read_csv(args.test)
    sample_df = pd.read_csv(args.sample)

    if "id" not in test_df.columns or "feature_16" not in test_df.columns:
        raise ValueError(f"{args.test} must contain columns: id, feature_16")

    perfect = build_known_region_predictions(test_df)
    # Keep sample ID ordering exactly as required by Kaggle format.
    out = schema_aligned_submission(sample_df["id"], perfect)

    if args.fallback is not None:
        fallback = load_submission(args.fallback)
        out, fallback_label = apply_fallback(out, fallback)
        fallback_label = str(args.fallback)
    else:
        out, fallback_label = apply_fallback(out, None)

    if args.output_name:
        output_name = args.output_name
    else:
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        output_name = f"submission_ZERO_{ts}.csv"

    args.output_dir.mkdir(parents=True, exist_ok=True)
    output_path = args.output_dir / output_name
    out.to_csv(output_path, index=False, float_format="%.18g")

    if not args.skip_train_check:
        train_df = pd.read_csv(args.train)
        validate_on_train(train_df)
```

```python

    counts = known_region_counts(perfect)
    print("\nDeterministic overwrite coverage on test:")
    print(f"  target_short  non-NaN from leak: {counts['target_short']} / {len(perfect)}")
    print(f"  target_medium non-NaN from leak: {counts['target_medium']} / {len(perfect)}")
    print(f"  target_long   non-NaN from leak: {counts['target_long']} / {len(perfect)}")
    print(f"  Tail fallback source: {fallback_label}")
    print(f"  Naming convention (docs/examples): {SUBMISSION_NAMING_CONVENTION}")
    print(f"\nSaved: {output_path}")


if __name__ == "__main__":
    main()
```

C.2 generate_tail_variants.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Generate active tail-variant submissions from deterministic leak base.

This script intentionally generates only the currently active variants:
- `zero`
- `expanding_all`

Submission naming convention for manual output names in docs/comments:
`submission_<PIPELINE>_<VARIANT>_CV<LOCAL_CV>.csv`
"""

from datetime import datetime
from pathlib import Path
import sys

import numpy as np
import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[1]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.constants import SUBMISSION_NAMING_CONVENTION
from trademaster_core.leak_math import build_known_region_predictions, patch_tail_from_future_f16
from trademaster_core.submission_io import save_submission
from trademaster_core.tail_models import compute_minute_mean_curve, future_from_minute_mean, future_zero

ACTIVE_METHODS = ("zero", "expanding_all")


def extrapolate_future_f16(
    f16: np.ndarray,
```

```python
    minute_id: np.ndarray,
    method: str,
    minute_mean_curve: np.ndarray | None = None,
    horizon: int = 240,
) -> np.ndarray:
    """Build 240-step future `feature_16` under the selected tail prior."""
    if method == "zero":
        return future_zero(horizon)

    if method == "expanding_all":
        if minute_mean_curve is None:
            raise ValueError("minute_mean_curve is required for method='expanding_all'")
        last_minute = int(minute_id[-1])
        return future_from_minute_mean(last_minute, minute_mean_curve, horizon)

    raise ValueError(f"Unknown method: {method}")


def main() -> None:
    """Entry point."""
    root = REPO_ROOT
    test_path = root / "data" / "raw" / "test_v2.csv"
    train_path = root / "data" / "raw" / "train_v2.csv"
    sample_path = root / "data" / "raw" / "sample_submission.csv"
    out_dir = root / "submissions"
    out_dir.mkdir(parents=True, exist_ok=True)

    test_df = pd.read_csv(test_path, usecols=["id", "minute_id", "feature_16"])
    train_df = pd.read_csv(train_path, usecols=["minute_id", "feature_16"])
    sample_df = pd.read_csv(sample_path, usecols=["id"])

    leak_base = build_known_region_predictions(test_df)
    minute_mean_curve = compute_minute_mean_curve(train_df)
    f16 = test_df["feature_16"].to_numpy(dtype="float64")
    minute_id = test_df["minute_id"].to_numpy(dtype="int64")

    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    generated: list[Path] = []

    for method in ACTIVE_METHODS:
        future = extrapolate_future_f16(
            f16=f16,
            minute_id=minute_id,
            method=method,
            minute_mean_curve=minute_mean_curve,
        )
        pred = patch_tail_from_future_f16(test_df=test_df, known_pred=leak_base, future=future)
        out_path = out_dir / f"submission_TAILVAR_{method.upper()}_{ts}.csv"
        save_submission(out_path, sample_df["id"], pred)
        generated.append(out_path)

```

```python
    print("Generated tail-variant submissions:")
    for p in generated:
        print(p)
    print(f"Naming convention (docs/examples): {SUBMISSION_NAMING_CONVENTION}")


if __name__ == "__main__":
    main()
```

C.3 generate_tailvar_macro_specialist.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Generate macro-specialist tail submission.

Macro specialist fits a ridge model that maps previous-day aggregate features
to the next-day 240-minute `feature_16` curve, then patches unknown tail rows.

Submission naming convention for manual output names in docs/comments:
`submission_<PIPELINE>_<VARIANT>_CV<LOCAL_CV>.csv`
"""

import argparse
import sys
from datetime import datetime
from pathlib import Path

import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[1]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.constants import SUBMISSION_NAMING_CONVENTION, TARGET_COLS
from trademaster_core.leak_math import build_known_region_predictions, patch_tail_from_future_f16
from trademaster_core.submission_io import save_submission
from trademaster_core.tail_models import (
    build_train_day_curves,
    fit_macro_nextday_ridge,
    predict_future_f16_macro,
)


def parse_args() -> argparse.Namespace:
    """Parse command-line arguments."""
    p = argparse.ArgumentParser(description="Generate tailvar submission with macro next-day specialist.")
    p.add_argument("--ridge-lambda", type=float, default=1.0, help="L2 regularization for ridge fit.")
    p.add_argument("--clip-abs", type=float, default=0.1, help="Clamp predicted future f16 to +/-clip-abs.")
```

```python
    p.add_argument(
        "--output-name",
        type=str,
        default="",
        help=(
            "Optional output filename. Recommended format: "
            "submission_TAILVAR_MACRO_SPECIALIST_LAM<val>_CV<LOCAL_CV>.csv. "
            "If omitted, script auto-generates one in submissions/."
        ),
    )
    return p.parse_args()


def main() -> None:
    """Entry point."""
    args = parse_args()

    root = REPO_ROOT
    train_path = root / "data" / "raw" / "train_v2.csv"
    test_path = root / "data" / "raw" / "test_v2.csv"
    sample_path = root / "data" / "raw" / "sample_submission.csv"
    out_dir = root / "submissions"
    out_dir.mkdir(parents=True, exist_ok=True)

    train_df = pd.read_csv(train_path, usecols=["date_id", "minute_id", "feature_16"])
    test_df = pd.read_csv(test_path, usecols=["id", "date_id", "minute_id", "feature_16"])
    sample_df = pd.read_csv(sample_path, usecols=["id"])

    perfect = build_known_region_predictions(test_df)
    day_curves = build_train_day_curves(train_df)
    beta, n_transitions = fit_macro_nextday_ridge(day_curves, ridge_lambda=args.ridge_lambda)
    future = predict_future_f16_macro(test_df, beta=beta, clip_abs=args.clip_abs)
    pred = patch_tail_from_future_f16(test_df, perfect, future=future)

    if args.output_name:
        out_path = out_dir / args.output_name
    else:
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        lam = f"{args.ridge_lambda:g}".replace(".", "p")
        out_path = out_dir / f"submission_TAILVAR_MACRO_SPECIALIST_LAM{lam}_{ts}.csv"

    save_submission(out_path, sample_df["id"], pred[["id", *TARGET_COLS]])

    print(f"Saved: {out_path}")
    print(f"Train day transitions used: {n_transitions}")
    print(f"Ridge lambda: {args.ridge_lambda}")
    print(f"Clip abs: {args.clip_abs}")
    print(f"Naming convention (docs/examples): {SUBMISSION_NAMING_CONVENTION}")


```

```python
if __name__ == "__main__":
    main()
```

C.4 generate_tailvar_robust_anchor.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Generate robust-anchor tail submission.

Robust-anchor = weighted blend of:
- macro specialist future (ridge next-day curve)
- expanding minute-mean future

Submission naming convention for manual output names in docs/comments:
`submission_<PIPELINE>_<VARIANT>_CV<LOCAL_CV>.csv`
"""

import argparse
import sys
from datetime import datetime
from pathlib import Path

import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[1]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.constants import SUBMISSION_NAMING_CONVENTION, TARGET_COLS
from trademaster_core.leak_math import build_known_region_predictions, patch_tail_from_future_f16
from trademaster_core.submission_io import save_submission
from trademaster_core.tail_models import (
    build_train_day_curves,
    compute_minute_mean_curve,
    fit_macro_nextday_ridge,
    future_from_minute_mean,
    predict_future_f16_macro,
)


def parse_args() -> argparse.Namespace:
    """Parse command-line arguments."""
    p = argparse.ArgumentParser(description="Generate robust-anchor tailvar submission (macro+expanding blend).")
    p.add_argument("--macro-weight", type=float, default=0.75, help="Weight on macro future (0..1).")
    p.add_argument("--ridge-lambda", type=float, default=0.1, help="Ridge lambda for macro specialist.")
    p.add_argument("--clip-abs", type=float, default=0.1, help="Clamp macro future to +/- clip_abs.")
    p.add_argument(
        "--output-name",
        type=str,
        default="",
```

```python
        help=(
            "Optional output filename. Recommended format: "
            "submission_TAILVAR_ROBUST_ANCHOR_W<val>_LAM<val>_CV<LOCAL_CV>.csv"
        ),
    )
    return p.parse_args()


def main() -> None:
    """Entry point."""
    args = parse_args()
    if not (0.0 <= args.macro_weight <= 1.0):
        raise ValueError("--macro-weight must be between 0 and 1")

    root = REPO_ROOT
    train_path = root / "data" / "raw" / "train_v2.csv"
    test_path = root / "data" / "raw" / "test_v2.csv"
    sample_path = root / "data" / "raw" / "sample_submission.csv"
    out_dir = root / "submissions"
    out_dir.mkdir(parents=True, exist_ok=True)

    train_df = pd.read_csv(train_path, usecols=["date_id", "minute_id", "feature_16"])
    test_df = pd.read_csv(test_path, usecols=["id", "date_id", "minute_id", "feature_16"])
    sample_df = pd.read_csv(sample_path, usecols=["id"])

    perfect = build_known_region_predictions(test_df)
    day_curves = build_train_day_curves(train_df)
    beta, _ = fit_macro_nextday_ridge(day_curves, ridge_lambda=args.ridge_lambda)
    fut_macro = predict_future_f16_macro(test_df, beta=beta, clip_abs=args.clip_abs)
    minute_mean_curve = compute_minute_mean_curve(train_df)
    fut_expand = future_from_minute_mean(int(test_df["minute_id"].iloc[-1]), minute_mean_curve)

    # Robust-anchor blend (requested):
    # future = w * macro + (1-w) * expanding_all
    w = float(args.macro_weight)
    future = w * fut_macro + (1.0 - w) * fut_expand

    pred = patch_tail_from_future_f16(test_df, perfect, future=future)

    if args.output_name:
        out_path = out_dir / args.output_name
    else:
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        w_tag = str(args.macro_weight).replace(".", "p")
        lam_tag = str(args.ridge_lambda).replace(".", "p")
        out_path = out_dir / f"submission_TAILVAR_ROBUST_ANCHOR_W{w_tag}_LAM{lam_tag}_{ts}.csv"

    save_submission(out_path, sample_df["id"], pred[["id", *TARGET_COLS]])

    print(f"Saved: {out_path}")
```

```python
    print(f"macro_weight={w:.6f}")
    print(f"expand_weight={1.0 - w:.6f}")
    print(f"weight_sum={(w + (1.0 - w)):.6f}")
    print(f"ridge_lambda={args.ridge_lambda}")
    print(f"clip_abs={args.clip_abs}")
    print(f"future_range=[{future.min():.8f}, {future.max():.8f}]")
    print(f"Naming convention (docs/examples): {SUBMISSION_NAMING_CONVENTION}")


if __name__ == "__main__":
    main()
```

C.5 Extracted baseline training blocks from solution.py

```python
# Extracted from notebooks/solution.py
# Included as baseline ML proof artifact for report appendix

# ===== BASELINE FUNCTION: train_champion_refinery =====
def train_champion_refinery(
    x_tr_refinery: np.ndarray,
    x_val_refinery: np.ndarray,
    x_test_refinery: np.ndarray,
    y_tr: np.ndarray,
    y_val: np.ndarray,
    test_ids: pd.Series,
    submissions_dir: Path,
) -> tuple[pd.DataFrame, float]:
    """Train the champion chained XGBoost model on refinery features."""
    print("     Training Champion XGBoost (Legal Features) [NaN-Safe Mode]...")

    x_tr_curr = x_tr_refinery.copy()
    x_val_curr = x_val_refinery.copy()
    x_test_curr = x_test_refinery.copy()

    final_preds = np.zeros((len(x_test_curr), 3))
    cv_scores: list[float] = []

    for i in range(3):
        print(f"        Target {i}...")
        honest_feat = None
        if i < 2:
            honest_feat = np.full(len(x_tr_curr), np.nan)
            tscv = TimeSeriesSplit(n_splits=5)
            for t_idx, v_idx in tscv.split(x_tr_curr):
                m = xgb.train(CONFIG.XGB_PARAMS, xgb.DMatrix(x_tr_curr[t_idx], y_tr[t_idx, i]), num_boost_round=100)
                honest_feat[v_idx] = m.predict(xgb.DMatrix(x_tr_curr[v_idx]))

        dtrain = xgb.DMatrix(x_tr_curr, label=y_tr[:, i])
        dval = xgb.DMatrix(x_val_curr, label=y_val[:, i])
        model = xgb.train(
            CONFIG.XGB_PARAMS,
            dtrain,
```

```python
            num_boost_round=2000,
            evals=[(dval, "val")],
            early_stopping_rounds=50,
            verbose_eval=False,
        )

        score = model.best_score
        cv_scores.append(score)
        print(f"          Target {i} Best Score: {score:.6f}")

        test_pred = model.predict(xgb.DMatrix(x_test_curr))
        val_pred = model.predict(dval)
        final_preds[:, i] = test_pred

        if i < 2 and honest_feat is not None:
            x_tr_curr = np.hstack([x_tr_curr, honest_feat.reshape(-1, 1)])
            x_val_curr = np.hstack([x_val_curr, val_pred.reshape(-1, 1)])
            x_test_curr = np.hstack([x_test_curr, test_pred.reshape(-1, 1)])

    sub = pd.DataFrame({"id": test_ids})
    sub["target_short"] = final_preds[:, 0]
    sub["target_medium"] = final_preds[:, 1]
    sub["target_long"] = final_preds[:, 2]
    sub = center_submission_targets(sub)

    avg_cv = weighted_cv_to_display(cv_scores)
    # Keep timestamp for run uniqueness; docs/examples use SUBMISSION_NAMING_CONVENTION.
    filename = f"submission_XGB_Refinery_CV{avg_cv:.5f}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    output_path = submissions_dir / filename
    sub.to_csv(output_path, index=False)
    print(f"     SAVED: {output_path}")
    return sub, avg_cv



# ===== BASELINE FUNCTION: train_purist =====
def train_purist(
    x_tr_scaled: np.ndarray,
    x_val_scaled: np.ndarray,
    x_test_scaled: np.ndarray,
    y_tr: np.ndarray,
    y_val: np.ndarray,
    test_ids: pd.Series,
    submissions_dir: Path,
) -> tuple[pd.DataFrame, float]:
    """Train the alternative 'purist' XGBoost model."""
    print("        Training 'Purist' Model (Base Features Only)...")
    final_preds = np.zeros((len(x_test_scaled), 3))
    cv_scores: list[float] = []

    for i in range(3):
        dtrain = xgb.DMatrix(x_tr_scaled, label=y_tr[:, i])
        dval = xgb.DMatrix(x_val_scaled, label=y_val[:, i])
```

```python
        params = CONFIG.XGB_PARAMS.copy()
        params["max_depth"] = 4
        model = xgb.train(
            params,
            dtrain,
            num_boost_round=1500,
            evals=[(dval, "val")],
            early_stopping_rounds=50,
            verbose_eval=False,
        )
        cv_scores.append(model.best_score)
        print(f"          Target {i} Best Score: {model.best_score:.6f}")
        final_preds[:, i] = model.predict(xgb.DMatrix(x_test_scaled))

    sub = pd.DataFrame({"id": test_ids})
    sub["target_short"] = final_preds[:, 0]
    sub["target_medium"] = final_preds[:, 1]
    sub["target_long"] = final_preds[:, 2]
    sub = center_submission_targets(sub)

    avg_cv = weighted_cv_to_display(cv_scores)
    # Keep timestamp for run uniqueness; docs/examples use SUBMISSION_NAMING_CONVENTION.
    filename = f"submission_XGB_Purist_CV{avg_cv:.5f}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    output_path = submissions_dir / filename
    sub.to_csv(output_path, index=False)
    print(f"           Saved Purist: {filename}")
    return sub, avg_cv



# ===== BASELINE FUNCTION: train_robust =====
def train_robust(
    x_tr_refinery: np.ndarray,
    x_val_refinery: np.ndarray,
    x_test_refinery: np.ndarray,
    y_tr: np.ndarray,
    y_val: np.ndarray,
    test_ids: pd.Series,
    submissions_dir: Path,
) -> tuple[pd.DataFrame, float]:
    """Train the alternative 'robust' XGBoost model."""
    print("           Training 'Robust' Model (Deep Trees)...")
    final_preds = np.zeros((len(x_test_refinery), 3))
    cv_scores: list[float] = []

    params = CONFIG.XGB_PARAMS.copy()
    params["max_depth"] = 8
    params["learning_rate"] = 0.01
    params["subsample"] = 0.6

    for i in range(3):
        dtrain = xgb.DMatrix(x_tr_refinery, label=y_tr[:, i])
        dval = xgb.DMatrix(x_val_refinery, label=y_val[:, i])
        model = xgb.train(
            params,
```

```
            dtrain,
            num_boost_round=3000,
            evals=[(dval, "val")],
            early_stopping_rounds=100,
            verbose_eval=False,
        )
        cv_scores.append(model.best_score)
        print(f"          Target {i} Best Score: {model.best_score:.6f}")
        final_preds[:, i] = model.predict(xgb.DMatrix(x_test_refinery))

    sub = pd.DataFrame({"id": test_ids})
    sub["target_short"] = final_preds[:, 0]
    sub["target_medium"] = final_preds[:, 1]
    sub["target_long"] = final_preds[:, 2]
    sub = center_submission_targets(sub)

    avg_cv = weighted_cv_to_display(cv_scores)
    # Keep timestamp for run uniqueness; docs/examples use SUBMISSION_NAMING_CONVENTION.
    filename = f"submission_XGB_Robust_CV{avg_cv:.5f}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    output_path = submissions_dir / filename
    sub.to_csv(output_path, index=False)
    print(f"           Saved Robust: {filename}")
    return sub, avg_cv



# ===== BASELINE FUNCTION: blend_refinery_and_robust =====
def blend_refinery_and_robust(
    sub_refinery: pd.DataFrame,
    sub_robust: pd.DataFrame | None,
    cv_ref: float | None,
    cv_rob: float | None,
    submissions_dir: Path,
) -> Path:
    """Build the final 50/50 ensemble from refinery and robust submissions."""
    print("       MIXING ENSEMBLE: 50% Refinery + 50% Robust...")

    if sub_robust is None:
        robust_files = glob.glob(str(submissions_dir / "submission_XGB_Robust_*.csv"))
        if not robust_files:
            raise ValueError("       Missing Robust predictions. Run robust training or ensure robust CSV exists.")
        latest_rob = max(robust_files, key=os.path.getctime)
        print(f"        Loaded Robust from disk: {latest_rob}")
        sub_robust = pd.read_csv(latest_rob)

    w_ref = 0.5
    w_rob = 0.5
    cols = ["target_short", "target_medium", "target_long"]
    sub_ens = pd.DataFrame({"id": sub_refinery["id"]})
    for c in cols:
        sub_ens[c] = sub_refinery[c] * w_ref + sub_robust[c] * w_rob

```

```python
    cv_ref_val = cv_ref if cv_ref is not None else 0.67309
    cv_rob_val = cv_rob if cv_rob is not None else 0.67037
    ens_cv = cv_ref_val * w_ref + cv_rob_val * w_rob

    # Keep timestamp for run uniqueness; docs/examples use SUBMISSION_NAMING_CONVENTION.
    filename = f"submission_Ensemble_Ref50_Rob50_CV{ens_cv:.5f}_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
    output_path = submissions_dir / filename
    sub_ens.to_csv(output_path, index=False)

    print(f"     SAVED ENSEMBLE: {filename}")
    print(f"   (Weights: Refinery {w_ref} | Robust {w_rob})")
    return output_path
```

C.6 Archival baseline: archival_neural_baseline.py

```python
#!/usr/bin/env python3

from __future__ import annotations

import argparse
import copy
import gc
import os
from pathlib import Path

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.cluster import KMeans
from sklearn.model_selection import KFold
from sklearn.preprocessing import RobustScaler
from torch.utils.data import DataLoader, TensorDataset


TARGET_COLS = ["target_short", "target_medium", "target_long"]
VIP_FEATURES = ["feature_19", "feature_5", "feature_27", "feature_2", "feature_13"]
SEEDS = [42, 43, 44, 101, 777]
N_FOLDS = 5
EPOCHS = 25
BATCH_SIZE = 2048
LEARNING_RATE = 5e-4
TARGET_SCALE = 100.0
KMEANS_CLUSTERS = 7
KMEANS_SAMPLE_STRIDE = 10
LAGS = [1, 2, 3, 5]
ROLL_WINDOWS = [5, 10, 20]
OUTPUT_NAME = "submission_resnet_honest.csv"


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")


```

```python
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Regenerate the archival neural-network baseline submission. "
            "Default output: submissions/submission_resnet_honest.csv"
        )
    )
    parser.add_argument(
        "--output-name",
        default=OUTPUT_NAME,
        help="Submission filename written into the project submissions folder.",
    )
    return parser.parse_args()


def resolve_paths() -> tuple[Path, Path]:
    project_root = Path(__file__).resolve().parents[1]

    if Path("/kaggle/input").exists():
        search_path = Path("/kaggle/input")
        for root, _dirs, files in os.walk(search_path):
            if "train_v2.csv" in files:
                data_path = Path(root)
                break
        else:
            raise FileNotFoundError("Could not locate train_v2.csv under /kaggle/input")
    else:
        data_path = project_root / "data" / "raw"

    return project_root, data_path


def load_raw_data(data_path: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
    print("   Loading raw train/test data...")
    train_df = pd.read_csv(data_path / "train_v2.csv")
    test_df = pd.read_csv(data_path / "test_v2.csv")

    # Sorting is necessary because all engineered lags and rolling windows are
    # defined in day/minute order.
    train_df = train_df.sort_values(["date_id", "minute_id"]).reset_index(drop=True)
    test_df = test_df.sort_values(["date_id", "minute_id"]).reset_index(drop=True)
    return train_df, test_df


def engineer_safe_features(df: pd.DataFrame, base_features: list[str]) -> pd.DataFrame:
    """Build the original notebook's "safe mode" NN feature set."""
    df_eng = df.copy()

    for col in base_features:
        grouped = df_eng.groupby("date_id")[col]
```

```python
        for lag in LAGS:
            df_eng[f"{col}_lag{lag}"] = grouped.shift(lag)

    for window in ROLL_WINDOWS:
        for col in VIP_FEATURES:
            grouped = df_eng.groupby("date_id")[col]
            df_eng[f"{col}_mean_{window}"] = grouped.transform(
                lambda x: x.rolling(window).mean()
            )
            df_eng[f"{col}_std_{window}"] = grouped.transform(
                lambda x: x.rolling(window).std()
            )

    # The original notebook ranked each value only against a recent rolling
    # context rather than against the entire day.
    for col in VIP_FEATURES:
        grouped = df_eng.groupby("date_id")[col]
        df_eng[f"{col}_rank"] = grouped.transform(
            lambda x: x.rolling(window=60, min_periods=10).rank(pct=True)
        ) - 0.5

    return df_eng


def build_feature_matrices(
    train_df: pd.DataFrame, test_df: pd.DataFrame
) -> tuple[np.ndarray, np.ndarray, np.ndarray, list[str]]:
    ignore = ["id", "date_id", "minute_id"] + TARGET_COLS
    base_features = [c for c in train_df.columns if c not in ignore]

    print("   Engineering archival NN features...")
    train_eng = engineer_safe_features(train_df, base_features)
    test_eng = engineer_safe_features(test_df, base_features)

    y = train_eng[TARGET_COLS].values
    final_cols = [c for c in train_eng.columns if c not in ignore]
    x_train = train_eng[final_cols].values
    x_test = test_eng[final_cols].values

    del train_eng, test_eng, train_df, test_df
    gc.collect()

    # Preserve notebook behavior: aggressively zero-fill missing/inf values.
    x_train = np.nan_to_num(x_train, nan=0.0, posinf=0.0, neginf=0.0)
    x_test = np.nan_to_num(x_test, nan=0.0, posinf=0.0, neginf=0.0)
    y = np.nan_to_num(y, nan=0.0)
    return x_train, x_test, y, final_cols


def scale_and_cluster(
    x_train: np.ndarray, x_test: np.ndarray
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    print("   Robust scaling features...")
    scaler = RobustScaler()
    x_train_scaled = scaler.fit_transform(x_train)
```

```python
    x_test_scaled = scaler.transform(x_test)

    print("   Generating KMeans market-state embeddings...")
    kmeans = KMeans(n_clusters=KMEANS_CLUSTERS, random_state=42, n_init=10)
    kmeans.fit(x_train_scaled[::KMEANS_SAMPLE_STRIDE])
    train_clusters = kmeans.predict(x_train_scaled)
    test_clusters = kmeans.predict(x_test_scaled)
    return x_train_scaled, x_test_scaled, train_clusters, test_clusters


class WeightedMAELoss(nn.Module):
    def __init__(self, weights: list[float] | None = None) -> None:
        super().__init__()
        if weights is None:
            weights = [0.5, 0.3, 0.2]
        self.weights = torch.tensor(weights, device=DEVICE)

    def forward(self, inputs: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        abs_err = torch.abs(inputs - targets)
        weighted_err = torch.sum(abs_err * self.weights, dim=1)
        return torch.mean(weighted_err)


class ResidualBlock(nn.Module):
    def __init__(self, hidden_dim: int, dropout: float) -> None:
        super().__init__()
        self.block = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.block(x)


class SniperResNet(nn.Module):
    def __init__(self, num_inputs: int, hidden_dim: int = 512, output_dim: int = 3, dropout: float = 0.25) -> None:
        super().__init__()
        self.embedding = nn.Embedding(KMEANS_CLUSTERS, 16)
        self.entry = nn.Sequential(
            nn.Linear(num_inputs, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.SiLU(),
            nn.Dropout(dropout),
        )
        self.res_blocks = nn.Sequential(
            ResidualBlock(hidden_dim, dropout),
            ResidualBlock(hidden_dim, dropout),
            ResidualBlock(hidden_dim, dropout),
        )
        self.merge = nn.Sequential(
            nn.Linear(hidden_dim + 16, hidden_dim // 2),
            nn.BatchNorm1d(hidden_dim // 2),
```

```
            nn.SiLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim // 2, output_dim),
        )

    def forward(self, x_num: torch.Tensor, x_cat: torch.Tensor) -> torch.Tensor:
        emb = self.embedding(x_cat)
        x = self.entry(x_num)
        x = self.res_blocks(x)
        concat = torch.cat([x, emb], dim=1)
        return self.merge(concat)


def train_ensemble(
    x_train_scaled: np.ndarray,
    x_test_scaled: np.ndarray,
    train_clusters: np.ndarray,
    test_clusters: np.ndarray,
    y: np.ndarray,
) -> np.ndarray:
    print(f"Training archival ResNet on {DEVICE}...")

    x_num_tensor = torch.FloatTensor(x_train_scaled).to(DEVICE)
    x_test_num_tensor = torch.FloatTensor(x_test_scaled).to(DEVICE)
    cluster_train_tensor = torch.LongTensor(train_clusters).to(DEVICE)
    cluster_test_tensor = torch.LongTensor(test_clusters).to(DEVICE)
    y_tensor = torch.FloatTensor(y * TARGET_SCALE).to(DEVICE)

    ensemble_preds = np.zeros((len(x_test_scaled), len(TARGET_COLS)))

    for seed in SEEDS:
        print(f"   Seed {seed}...")
        kfold = KFold(n_splits=N_FOLDS, shuffle=True, random_state=seed)
        seed_preds = np.zeros((len(x_test_scaled), len(TARGET_COLS)))

        for train_idx, val_idx in kfold.split(x_num_tensor, y_tensor):
            train_ds = TensorDataset(
                x_num_tensor[train_idx],
                cluster_train_tensor[train_idx],
                y_tensor[train_idx],
            )
            val_ds = TensorDataset(
                x_num_tensor[val_idx],
                cluster_train_tensor[val_idx],
                y_tensor[val_idx],
            )

            train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
            val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE * 2, shuffle=False)

            model = SniperResNet(num_inputs=x_num_tensor.shape[1]).to(DEVICE)
            criterion = WeightedMAELoss(weights=[0.5, 0.3, 0.2])
            optimizer = optim.AdamW(
```

```
                model.parameters(),
                lr=LEARNING_RATE,
                weight_decay=1e-4,
            )
            scheduler = optim.lr_scheduler.OneCycleLR(
                optimizer,
                max_lr=LEARNING_RATE,
                steps_per_epoch=len(train_loader),
                epochs=EPOCHS,
            )

            best_wmae = float("inf")
            best_weights = copy.deepcopy(model.state_dict())

            for _epoch in range(EPOCHS):
                model.train()
                for bx_num, bx_cat, by in train_loader:
                    optimizer.zero_grad()
                    out = model(bx_num, bx_cat)
                    loss = criterion(out, by)
                    loss.backward()
                    optimizer.step()
                    scheduler.step()

                model.eval()
                val_loss = 0.0
                with torch.no_grad():
                    for bx_num, bx_cat, by in val_loader:
                        out = model(bx_num, bx_cat)
                        val_loss += criterion(out, by).item() * bx_num.
                            size(0)

                avg_wmae = val_loss / len(val_ds)
                if avg_wmae < best_wmae:
                    best_wmae = avg_wmae
                    best_weights = copy.deepcopy(model.state_dict())

            model.load_state_dict(best_weights)
            model.eval()

            with torch.no_grad():
                fold_outputs = []
                test_loader = DataLoader(
                    TensorDataset(x_test_num_tensor,
                        cluster_test_tensor),
                    batch_size=4096,
                )
                for bx_num, bx_cat in test_loader:
                    fold_outputs.append(model(bx_num, bx_cat).cpu().
                        numpy())
                seed_preds += np.concatenate(fold_outputs) / N_FOLDS

            del model, optimizer
            if torch.cuda.is_available():
                torch.cuda.empty_cache()

        ensemble_preds += seed_preds / len(SEEDS)
        print(f"      completed seed {seed}")
```

```python

    return ensemble_preds / TARGET_SCALE


def save_submission(data_path: Path, output_path: Path, preds: np.ndarray) -> None:
    submission = pd.read_csv(data_path / "test_v2.csv", usecols=["id"])
    submission["target_short"] = preds[:, 0]
    submission["target_medium"] = preds[:, 1]
    submission["target_long"] = preds[:, 2]

    # Preserve notebook behavior: center each target around zero before export.
    for col in TARGET_COLS:
        submission[col] = submission[col] - submission[col].mean()

    output_path.parent.mkdir(parents=True, exist_ok=True)
    submission.to_csv(output_path, index=False)
    print(f"Saved archival NN submission to {output_path}")


def main() -> None:
    args = parse_args()
    project_root, data_path = resolve_paths()
    output_path = project_root / "submissions" / args.output_name

    print(f"Initializing archival neural baseline on {DEVICE}...")
    train_df, test_df = load_raw_data(data_path)
    x_train, x_test, y, _feature_cols = build_feature_matrices(train_df, test_df)
    x_train_scaled, x_test_scaled, train_clusters, test_clusters = scale_and_cluster(
        x_train, x_test
    )
    preds = train_ensemble(
        x_train_scaled, x_test_scaled, train_clusters, test_clusters, y
    )
    save_submission(data_path, output_path, preds)


if __name__ == "__main__":
    main()
```

C.7 Legacy helper: verify_minute27_cutoff.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Verify why minute-27 is used as the aligned boundary cutoff.

The script inspects test size/phase and reports the cycle alignment that
motivates minute-27 backtest boundaries.
"""

from pathlib import Path
import sys
```

```python

import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[2]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.paths import find_repo_root


def main() -> None:
    """Load test index structure and print phase-alignment diagnostics.
    """
    root = find_repo_root(Path(__file__))
    test = pd.read_csv(root / "data" / "raw" / "test_v2.csv", usecols=[
        "date_id", "minute_id"])

    n = len(test)
    first_date = int(test.iloc[0]["date_id"])
    first_minute = int(test.iloc[0]["minute_id"])
    last_date = int(test.iloc[-1]["date_id"])
    last_minute = int(test.iloc[-1]["minute_id"])

    counts = test.groupby("date_id").size()
    partial_days = counts[counts != 240]

    # Global phase position in a 240-step day cycle.
    # This is the alignment used when leakage logic is treated as one
        continuous series.
    phase_last = (n - 1) % 240
    phase_next = n % 240

    # Check continuity model: minute at index i should be (first_minute
        + i) % 240.
    minute_from_phase_last = (first_minute + phase_last) % 240
    minute_from_phase_next = (first_minute + phase_next) % 240

    print("=== Test Cutoff Verification ===")
    print(f"rows: {n}")
    print(f"first row: date={first_date}, minute={first_minute}")
    print(f"last row : date={last_date}, minute={last_minute}")
    print(f"rows % 240 = {n % 240}")
    print(f"(rows-1) % 240 = {phase_last}")
    print()
    print("Partial day structure:")
    print(partial_days.to_dict())
    print()
    print("Phase alignment checks:")
    print(f"minute from phase_last = (first_minute + phase_last) % 240
        = {minute_from_phase_last}")
    print(f"minute from phase_next = (first_minute + phase_next) % 240
        = {minute_from_phase_next}")
    print()
    print("Interpretation:")
    print("- Raw test endpoint minute_id is 239.")
    print("- Equivalent cycle phase at endpoint is 27.")
    print("- Therefore minute-27 boundary simulation is the correct
        aligned cutoff for historical backtests.")
```

```python


if __name__ == "__main__":
    main()
```

C.8 Legacy helper: compare_boundary_cv.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Compare boundary-simulation CV at cutoff minute 27 vs 239.

This helper ranks simple tail priors (zero/AR1/AR5) under both boundary
definitions and prints side-by-side MAE statistics.
"""

from pathlib import Path
import sys

import numpy as np
import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[2]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

from trademaster_core.paths import find_repo_root


def fut_zero(_: int, f16: np.ndarray) -> np.ndarray:
    """Future prior: hard zeros."""
    return np.zeros(240, dtype="float64")


def fut_ar1(e: int, f16: np.ndarray) -> np.ndarray:
    """Future prior: clipped AR(1) recursion fitted on recent history."
        ""
    hist = f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-4000:] if len(hist) > 4000 else hist
    if len(hist) < 20:
        return np.zeros(240, dtype="float64")

    denom = float(np.dot(hist[:-1], hist[:-1]))
    a = float(np.dot(hist[1:], hist[:-1]) / denom) if denom > 1e-12
        else 0.0
    a = float(np.clip(a, -0.995, 0.995))

    out = np.zeros(240, dtype="float64")
    cur = float(hist[-1])
    for i in range(240):
        cur = a * cur
        out[i] = cur
    return out


def fut_ar5(e: int, f16: np.ndarray) -> np.ndarray:
```

```python
    """Future prior: clipped AR(5) recursion fitted by least squares."""
    hist = f16[: e + 1]
    hist = hist[np.isfinite(hist)]
    hist = hist[-6000:] if len(hist) > 6000 else hist
    p = 5
    if len(hist) < p + 20:
        return np.zeros(240, dtype="float64")

    y = hist[p:]
    x = np.column_stack([np.ones(len(y))] + [hist[p - i - 1 : len(hist) - i - 1] for i in range(p)])
    try:
        beta, *_ = np.linalg.lstsq(x, y, rcond=None)
    except np.linalg.LinAlgError:
        return np.zeros(240, dtype="float64")

    buf = list(hist[-p:])
    out = np.zeros(240, dtype="float64")
    for i in range(240):
        row = np.array([1.0] + buf[::-1], dtype="float64")
        nxt = float(row @ beta)
        nxt = float(np.clip(nxt, -0.1, 0.1))
        out[i] = nxt
        buf = buf[1:] + [nxt]
    return out


def boundary_score_for_future(tr: pd.DataFrame, f16: np.ndarray, e: int, fut: np.ndarray) -> float:
    """Compute competition-style weighted boundary MAE for one day endpoint."""
    err_short = []
    for i in range(e - 9, e + 1):
        err_short.append(abs(tr.at[i, "target_short"] - fut[i + 10 - e - 1]))

    err_medium = []
    for i in range(e - 59, e + 1):
        vals = []
        for k in range(1, 7):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_medium.append(abs(tr.at[i, "target_medium"] - pred))

    err_long = []
    for i in range(e - 239, e + 1):
        vals = []
        for k in range(1, 25):
            j = i + 10 * k
            vals.append(f16[j] if j <= e else fut[j - e - 1])
        pred = np.prod(1 + np.array(vals)) - 1
        err_long.append(abs(tr.at[i, "target_long"] - pred))

    return 0.5 * np.mean(err_short) + 0.3 * np.mean(err_medium) + 0.2 * np.mean(err_long)

```

```python

def build_eval_indices(
    tr: pd.DataFrame, f16: np.ndarray, by_day: dict[int, np.ndarray], cutoff_minute: int
) -> list[int]:
    """Build valid day endpoints for a given cutoff minute."""
    eval_idx: list[int] = []
    for d, idx in by_day.items():
        if len(idx) <= cutoff_minute:
            continue

        e = int(idx[cutoff_minute])
        if e < 239:
            continue
        if e + 240 >= len(tr):
            continue

        pref = f16[idx[: cutoff_minute + 1]]
        if not np.all(np.isfinite(pref)):
            continue

        eval_idx.append(e)
    return eval_idx


def run_cutoff(
    tr: pd.DataFrame, f16: np.ndarray, by_day: dict[int, np.ndarray], cutoff_minute: int
) -> tuple[int, dict[str, dict[str, float]]]:
    """Evaluate all methods for a fixed cutoff minute."""
    methods = {
        "zero": fut_zero,
        "ar1": fut_ar1,
        "ar5": fut_ar5,
    }
    eval_idx = build_eval_indices(tr, f16, by_day, cutoff_minute)

    out: dict[str, dict[str, float]] = {}
    for name, fn in methods.items():
        scores = []
        for e in eval_idx:
            fut = fn(e, f16)
            s = boundary_score_for_future(tr, f16, e, fut)
            scores.append(s)
        arr = np.asarray(scores, dtype="float64")
        out[name] = {
            "mean_mae": float(arr.mean()),
            "median_mae": float(np.median(arr)),
            "std_mae": float(arr.std()),
        }
    return len(eval_idx), out


def main() -> None:
    """Entry point for cutoff-27 vs cutoff-239 comparison."""
    root = find_repo_root(Path(__file__))
    tr = pd.read_csv(root / "data" / "raw" / "train_v2.csv")
    f16 = tr["feature_16"].to_numpy(dtype="float64")
```

```python
    date = tr["date_id"].to_numpy()
    by_day = {int(d): np.where(date == d)[0] for d in np.unique(date)}

    rows = []
    print("Comparing cutoff minute 27 vs 239 on train boundary simulation...")
    for cutoff in [27, 239]:
        n_days, result = run_cutoff(tr, f16, by_day, cutoff)
        print(f"\ncutoff={cutoff} | valid_backtest_days={n_days}")
        for method, stats in result.items():
            print(
                f"  {method.upper():<4} mean={stats['mean_mae']:.8f} "
                f"median={stats['median_mae']:.8f} std={stats['std_mae']:.8f}"
            )
            rows.append(
                {
                    "cutoff": cutoff,
                    "method": method,
                    "valid_backtest_days": n_days,
                    "mean_mae": stats["mean_mae"],
                    "median_mae": stats["median_mae"],
                    "std_mae": stats["std_mae"],
                }
            )

    df = pd.DataFrame(rows)
    pivot = df.pivot(index="method", columns="cutoff", values="mean_mae")
    if 27 in pivot.columns and 239 in pivot.columns:
        pivot["delta_239_minus_27"] = pivot[239] - pivot[27]
    print("\nSide-by-side mean MAE:")
    print(pivot.sort_values(by=27).to_string(float_format=lambda x: f"{x:.8f}"))


if __name__ == "__main__":
    main()
```

C.9 Sanity checks: run_sanity_checks.py

```python
#!/usr/bin/env python3
from __future__ import annotations

"""Lightweight deterministic sanity checks for leakage and tail coverage."""

from pathlib import Path
import sys

import numpy as np
import pandas as pd

REPO_ROOT = Path(__file__).resolve().parents[2]
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))

```

```python
from trademaster_core.constants import TARGET_COLS
from trademaster_core.leak_math import build_known_region_predictions, compounded_target, validate_leak_mae
from trademaster_core.submission_io import schema_aligned_submission


def check_leak_equations(train_df: pd.DataFrame) -> None:
    """Verify deterministic target equations on scored-valid windows."""
    stats = validate_leak_mae(train_df)
    mae_short = stats["target_short"]
    mae_medium = stats["target_medium"]
    mae_long = stats["target_long"]

    assert mae_short <= 1e-12, f"target_short leak check failed: MAE={mae_short}"
    assert mae_medium <= 1e-11, f"target_medium leak check failed: MAE={mae_medium}"
    assert mae_long <= 1e-11, f"target_long leak check failed: MAE={mae_long}"

    print(f"[PASS] Leak equations: short={mae_short:.3e} medium={mae_medium:.3e} long={mae_long:.3e}")


def check_test_tail_coverage(test_df: pd.DataFrame) -> None:
    """Verify unknown tail sizes implied by shift/compound windows."""
    f16 = test_df["feature_16"]
    short_na = int(f16.shift(-10).isna().sum())
    medium_na = int(compounded_target(f16, 6).isna().sum())
    long_na = int(compounded_target(f16, 24).isna().sum())

    assert short_na == 10, f"Expected 10 short NaNs, got {short_na}"
    assert medium_na == 60, f"Expected 60 medium NaNs, got {medium_na}"
    assert long_na == 240, f"Expected 240 long NaNs, got {long_na}"

    print(f"[PASS] Test unknown tails: short={short_na} medium={medium_na} long={long_na}")


def check_submission_schema(sample_df: pd.DataFrame, test_df: pd.DataFrame) -> None:
    """Build leak-base frame and verify schema/order compatibility."""
    pred = build_known_region_predictions(test_df)
    out = schema_aligned_submission(sample_df["id"], pred)

    assert list(out.columns) == ["id", *TARGET_COLS], "Submission schema mismatch"
    assert out["id"].equals(sample_df["id"]), "Submission id ordering mismatch"
    assert len(out) == len(sample_df), "Submission length mismatch"

    print("[PASS] Submission schema/order check")


def check_known_inf_location(train_df: pd.DataFrame) -> None:
    """Ensure known Inf footprint remains limited to feature_2."""
    num = train_df.select_dtypes(include=["number"])
```

```python
    arr = num.to_numpy()
    rows, cols = np.where(np.isinf(arr))
    inf_cols = sorted(set(num.columns[cols]))
    assert inf_cols == ["feature_2"], f"Unexpected Inf columns: {inf_cols}"
    print(f"[PASS] Inf audit: columns={inf_cols}, count={len(rows)}")


def main() -> None:
    """Run all sanity checks."""
    root = REPO_ROOT
    train_df = pd.read_csv(root / "data" / "raw" / "train_v2.csv")
    test_df = pd.read_csv(root / "data" / "raw" / "test_v2.csv")
    sample_df = pd.read_csv(root / "data" / "raw" / "sample_submission.csv")

    check_leak_equations(train_df)
    check_test_tail_coverage(test_df)
    check_submission_schema(sample_df, test_df)
    check_known_inf_location(train_df)
    print("ALL SANITY CHECKS PASSED")


if __name__ == "__main__":
    main()
```